



COMPUTER  
SCIENCE SOCIETY

ST OLAVE'S  
GRAMMAR SCHOOL

---

# HYPERTEXT

---

ISSUE TWO • DECEMBER • 2020





# EDITOR'S NOTE

ISSUE TWO • DECEMBER • 2020

I am pleased to announce the second issue of HYPertext, the St Olave's Computer Science Journal, full of fascinating articles regarding a wide variety of topics, ranging from GPT-3 to fractals. Issue Two aims to give an insight into aspects of Computer Science beyond the school curriculum and displays the proactivity and passion that the writers of these articles have.

Feedback is welcomed! Please contact your computer science teacher, the Head of Computing at [rzeshan@saintolaves.net](mailto:rzeshan@saintolaves.net) or myself at [minjie.guo@saintolaves.net](mailto:minjie.guo@saintolaves.net) with questions you may have or articles you would like to contribute for the next edition.

Thank you to everyone that contributed articles and I hope there are many more issues to come.

---

**MIN JIE GUO**

**PRESIDENT OF THE COMPUTER SCIENCE SOCIETY,  
EDITOR OF THE COMPUTER SCIENCE JOURNAL**



# CONTENTS

GPT-3 AND NLP: HOW MACHINE  
LEARNING SYSTEMS MIMIC HUMAN  
CREATIVITY

GRACE SODUNKE PAGE 4

BEHIND THE SCENES OF  
AUTONOMOUS VEHICLES

HARDIV HARSHAKUMAR PAGE 8

DEEP LEARNING - POTENTIAL TO  
CHANGE THE FUTURE

PRADHAY AMARNATH PAGE 14

AUTOPILOT SYSTEMS

RYAN THUKALIL PAGE 17

ALGORITHMIC BIAS

ADVAIT MENON PAGE 20

5G: IS IT REALLY GOING TO KILL  
US ALL?

MIN JIE GUO PAGE 22

FRACTALS IN PROGRAMMING

JANET THOMAS ALBERT PAGE 26

THE BOMBE

JAMES PERRY PAGE 29

AN INTRODUCTION TO  
BLOCKCHAIN

ARJUN JAVAGAL PAGE 34

WHY YOU SHOULD LEARN TO  
CODE

NAVANEETH MADHAVAN PAGE 37

COMPUTER SCIENCE SOCIETY

MIN JIE GUO PAGE 41

# GPT-3 AND NLP: HOW MACHINE LEARNING SYSTEMS

## MIMIC HUMAN CREATIVITY

GRACE SODUNKE

Natural Language Processing is an interdisciplinary field of Artificial Intelligence that addresses the question: "Can machines understand and interact using human language?" This straightforward yet intractable problem can potentially yield many benefits for humanity, not limited to the automated generation of natural sounding text, as well as sophisticated chatbots and assistants. In fact, NLP technology already has important everyday applications including machine translation, speech recognition and sentiment analysis.

In May 2020, a major milestone occurred when OpenAI released its state-of-the-art Generative Pre-trained Transformer 3 (GPT-3). This immense language model was trained on trillions of words from the Internet which allows it to display remarkable ability in complex human tasks. A prominent example is its ability to generate fiction and non-fiction articles that are difficult to distinguish from those written by humans (Figure 1). Furthermore, this breakthrough model is task-agnostic and achieves strong performance using few-shot learning approaches. Unlike previous models, GPT-3 did not have to be fine-tuned using vast supervised datasets to complete a specific task after pre-training; by only providing a few examples of the task it is asked to do, it becomes extremely proficient at adapting to a new domain.

I will explore the current capabilities of GPT-3; the neural network architecture that has achieved this benchmark performance; the short-term and long-term societal impacts; and the subsequent breakthroughs that are necessary to achieve ideal capabilities.

### How it works:

Well-known abilities of GPT-3 currently include:

- Generating poetry and prose
- Text summarisation, question-answering, language translation, cloze tasks
- Generating program code using various programming languages, given only a natural language description of the program's function.

A key factor that enables this performance is the model's immense size of 175 billion parameters. Parameters refer to the weights that are optimised in a neural network during training. As a result, GPT-3 is ten times larger than the previous state-of-the-art model released in February, Microsoft's Turing NLG, which has 17 billion parameters and is limited to text summarisation and question-answering. This consequently supports the hypothesis that continuous scaling of language models is necessary to improve learning abilities.

GPT-3, as well as its predecessor GPT-2, implement transformer models. This is a deep learning model that uses attention mechanisms as well as an encoder-decoder architecture. It replaces LSTM recurrent neural networks (RNNs) since these keep limited internal memory about past data to provide context when processing each element of a sequence.

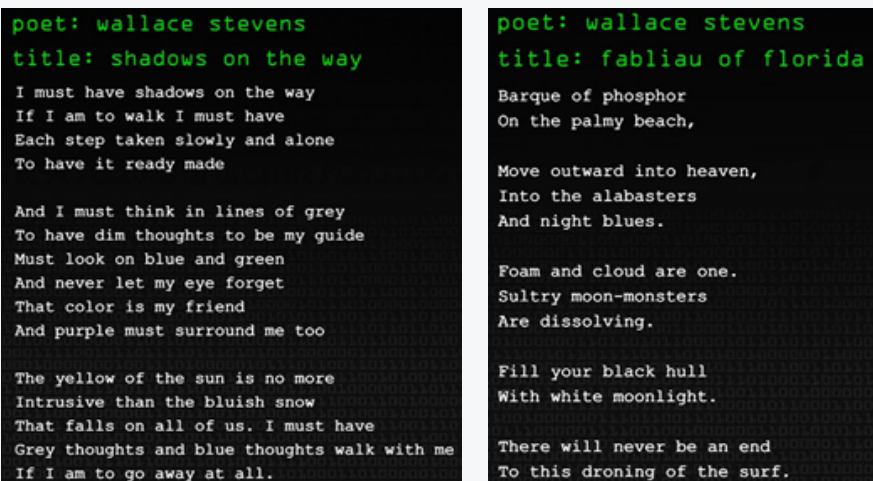


Figure 1: Which poem was written by GPT-3?

GAs a Generative Pre-Trained Transformer, GPT-3 was trained with an unsupervised dataset consisting of unlabelled data from which the model identified patterns. Billions of input tokens (individual words) within the pre-training dataset were sourced from large web corpora, including Wikipedia and a filtered version of public Common Crawl data.

Now a look into the mechanisms behind this fascinating architecture!

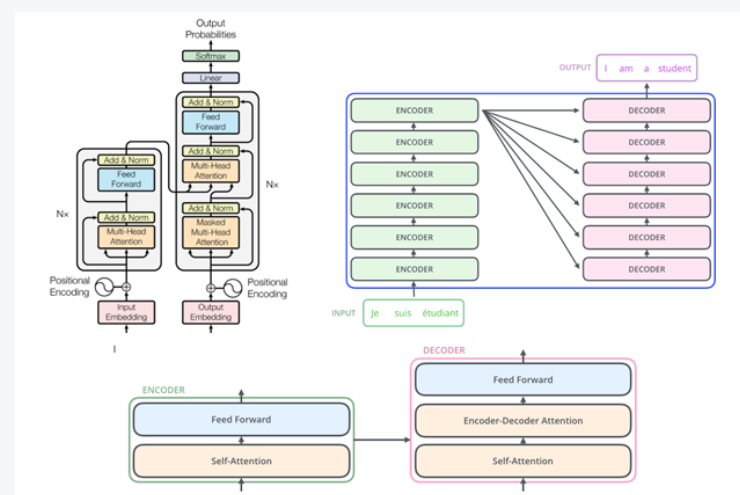


Figure 2: The Transformer Model architecture

A Transformer model includes an encoder and decoder: the encoder consists of encoding layers that iteratively process input and the decoder contains decoding layers that do the same to the encoder's output. Attention mechanisms, used in each encoding and decoding layer, emerged as an important innovation in the design of deep learning algorithms in 2017. They allow language models to efficiently use context when outputting text by only referring to relevant information earlier in the sequence. Thus each encoding module consists of a multi-head attention layer and a feed-forward neural network. In addition to these components, decoding modules contain an additional attention mechanism to focus on the relevant parts from the outputs of encodings (Figure 2).

Before entering the bottom-most encoder, an embedding algorithm is used to represent each input word as a vector. Positional encoding then involves adding a vector to each embedding to account for the order of words in a sentence.

Within the encoder, the multi-head attention layer computes several attention weights in parallel for each token in a sentence. To calculate one self-attention unit, the model learns three weight matrices that each produce a query vector, key vector and value vector when multiplied by the input word embedding. Then the dot product is calculated between an embedding's query vector and all other value vectors to find the attention weights between tokens. To stabilise the computed gradients, the attention weights are divided by the square root of the dimension of the key vectors. These results are passed through the softmax function which normalises the weights so that they are positive and sum to 1 (Figure 3). The attention units are multiplied by the corresponding value vectors and these weighted vectors are summed to produce the output for that layer (Figure 4). To achieve multi-headed attention, the attention mechanism is repeated with multiple sets of Q, K and V weight matrices and the weighted vectors are concatenated and multiplied by an additional matrix. This allows many levels of semantic relevance to be captured.

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Figure 3: Softmax function. To transform a vector of logits (unscaled values) into a probability distribution that sums to 1,  $e$  is raised to the power of each of the numbers and its fraction of the sum of the exponentials is calculated.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Figure 4: Calculating attention. The function is a vectorised implementation because matrix operations are more efficient to compute. The process of simultaneously calculating attention weights also lends itself well to parallel processing, therefore transformer models reduce training time.

The output encodings are passed to a feed-forward neural network. This is a common architecture used to optimise a function mapping the input values to output values by minimising the cost function. Layer normalisation steps both precede and follow this layer so that input and output values are on a common scale. The resulting matrix can be passed to the next encoder in the stack, of which there are around six. Following this, the output of encoders is used in an encoder-decoder attention layer within decoders.

The bottom-most decoder takes as input the positional encodings and embeddings of previous output by the decoder stack. Like in encoders, this is used in a multi-head attention module. Next, in the additional attention layer of each decoding unit, the output of the encoder stack is used to focus on relevant embeddings from the initial input sequence in order to generate part of the decoded output for this time step. They are used for the key and value vectors, while the query vectors come from the preceding decoder layer. Layer normalisation occurs around each of the sub layers like in the encoder. Moreover, a feed forward neural network is applied to each calculated attention unit.

The final step involves a linear layer that transforms the vector produced by the stack of decoders into a vector whose dimension corresponds to the number of words added to the model's vocabulary during training. The softmax function is then used to convert the scores into probabilities that sum to 1, and the word with the highest probability is the output for this time step.

The training methodology of GPT-3 exemplifies the trend towards developing systems with generalised abilities. Fine-tuning a model after pre-training involves sourcing hundreds of thousands of task-specific data examples for supervised learning. This traditional method has many drawbacks for language models as it limits their use across different tasks as well as their ability to generalise to new domains.

As a result, one main aim of creating GPT-3 was to achieve task-agnostic, high performance. This was achieved through few-shot learning, a ground-breaking mode of in-context learning in which the model recognises how to complete a task given only a few demonstrations. This largely reduces the amount of task specific data required and even allows GPT-3 to achieve 86.4% accuracy. (Figure 5)

Other settings for in-context learning are one-shot learning, when one example is provided at inference time, and zero-shot learning, when the model is given only a natural language description of the task. These settings are particularly significant because the quick reasoning that is assessed closely models how humans learn. With tasks such as performing arithmetic and solving anagrams, 76% accuracy was achieved in the zero-shot setting.

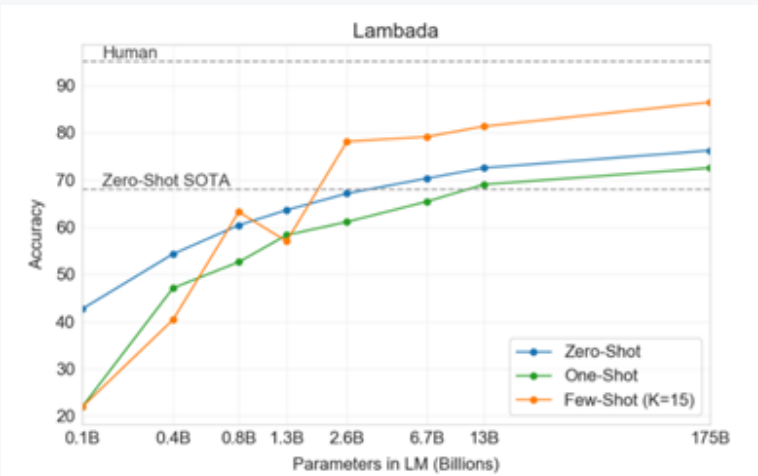


Figure 5: A measure of GPT-3's performance on common language tasks such as completing text and predicting the next word in a sentence, using the LAMBADA dataset. Few-shot learning allows the accuracy of language models increase significantly, achieving close to human performance. GPT-3 with 175 billion parameters achieves better performance than the previous state of the art, Turing NLG, by 18%.

## Current applications

OpenAI released a GPT-3 API in June, allowing anyone to explore short-term use cases for this technology. Recent projects allow users to converse with chatbots that can even imitate famous historical figures. News outlets such as the Guardian have also attempted to synthetically generate articles. There are many potential uses of GPT-3 in the long-term, one of which involves automatically generating code when the model is given a zero-shot description of the program's design. Moreover, integrating the model with speech-to-text technology could explore its application in virtual home assistants.

Despite this, many have drawn attention to harmful exploitations of this model. As GPT-3 can produce high-quality text given a prompt, a major risk is the spread of misinformation and this is easily propagated online. 44% of adults worldwide encountered fake news in the media in 2019. Moreover, cybersecurity risks include better-targeted phishing and social engineering attacks by Advanced Persistent Threats (APTs), since writing quality is currently a bottleneck. Should we be concerned that programs can indiscernibly imitate human-authored prose? Nevertheless, risk mitigation through threat analysis and the use of automatic discriminators has become a priority for future research.

## What is next?

GPT-3 has captured the imagination of many and is arguably 'one of the most interesting and important AI systems ever produced'. Ultimately, the goal is to have general language systems that can fluidly adapt and GPT-3's abilities indicate that large-scale models are necessary to facilitate this. Thus the conceptual limitations that remain are focused on algorithmic design. For example, bidirectional transformer models that encode using the context of words on the left and right are likely to improve zero-shot performance on the scale of GPT-3. As a result, unsupervised deep learning techniques that use training data more efficiently will allow us to explore the future potential of advanced AI systems.

## Bibliography:

Brown, Tom & Mann, Benjamin & Ryder, Nick & Subbiah, Melanie & Kaplan, Jared & Dhariwal, Prafulla & Neelakantan, Arvind & Shyam, Pranav & Sastry, Girish & Askell, Amanda & Agarwal, Sandhini & Herbert-Voss, Ariel & Krueger, Gretchen & Henighan, Tom & Child, Rewon & Ramesh, Aditya & Ziegler, Daniel & Wu, Jeffrey & Winter, Clemens & Amodei, Dario. (2020). Language Models are Few-Shot Learners.

Vaswani, Ashish & Shazeer, Noam & Parmar, Niki & Uszkoreit, Jakob & Jones, Llion & Gomez, Aidan & Kaiser, Lukasz & Polosukhin, Illia. (2017). Attention Is All You Need.

Alammar, J., (2020). The Illustrated Transformer. Available at: <http://jalammar.github.io/illustrated-transformer/>

Nature Machine Intelligence, (2020). Next chapter in artificial writing.

Computerphile, (2020). GPT-3: An Even Bigger Language Model. Available at: [https://www.youtube.com/watch?v=\\_8yVOC4ciXc](https://www.youtube.com/watch?v=_8yVOC4ciXc)

# BEHIND THE SCENES OF AUTONOMOUS VEHICLES

HARDIV HARSHAKUMAR

Autonomous Vehicles are undoubtedly an exciting prospect for the future. Just imagine being able to relax whilst you are driven by your high-tech chauffeur to the location of your choice. Not only that, but autonomous vehicles could potentially greatly reduce the price of delivery and transport. They may also make our roads more efficient, as CO2 emissions could be cut down by sharing fleets of self-driving vehicles. Studies and research suggest that these vehicles can help make roads safer for everyone by reducing fatality rates from human error, the cause of over 90% of car accidents.

But what makes it tick? It seems simple to tell a computer on wheels where it needs to go, but there's a lot of calculations and algorithms being processed, behind the scenes. These calculations need to determine how to:

- Get the vehicle to the location
- Get it there safely
- Get it there quickly enough

To carry out these tasks, the self-driving task can be decomposed into 5 main areas:

## 1. Computer vision

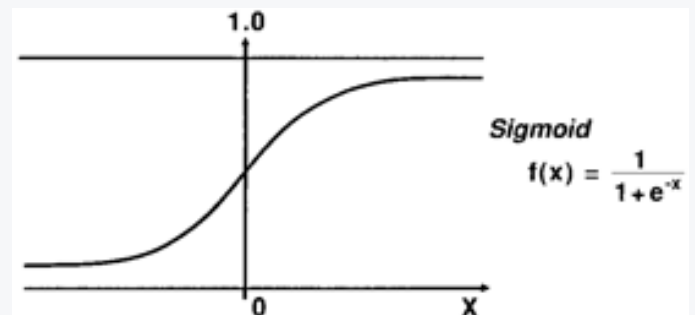
This refers to how the vehicle makes sense of the environment using cameras. This will help carry out tasks like lane finding, road curvature estimation and traffic light detection.

There are two main parts of computer vision:

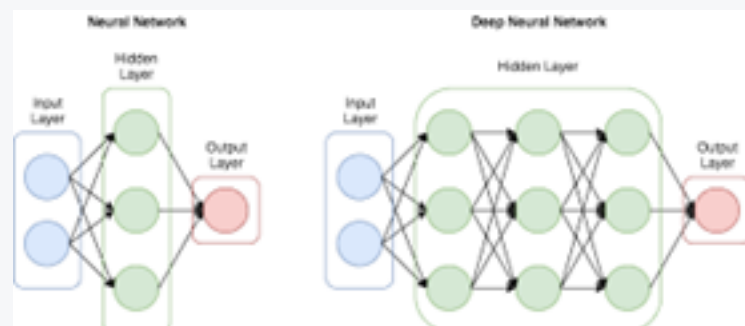
Detection - This is the process of finding out where an object is, relative to the vehicle. This is very important to consider as it will allow decisions, such as when the vehicle needs to come to a stop, to be made, ensuring the safety of passengers and pedestrians.

Classification - This is the process of finding out what an object is (image recognition), carried out using deep learning. In image recognition, neural networks are 'trained' on large sets of data - images in this case - for the network to eventually 'learn' the abstract representations of each image, and successfully be able to identify what those images represent.

Very basically, the way a neural network does this is by taking in inputs and storing them in nodes within the input layer. These inputs would be the colour values of the pixels within the image, converted into a value between 0 and 1, with the help of the sigmoid function, shown below.



The nodes in the input layer then cause a specific combination of nodes in the hidden layers to have higher values. This causes specific nodes in the output layer to have higher values than the others. The node in the output layer with the value closest to 1 is then considered as the answer/output.

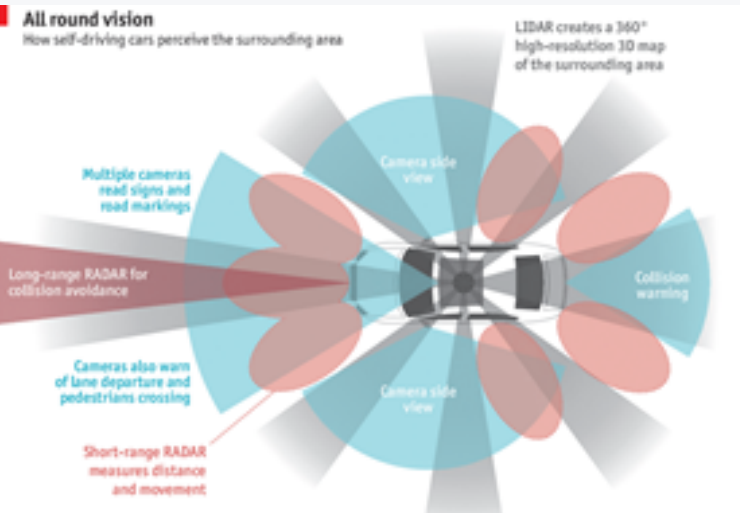


In a perfect world, these nodes would represent pixels that make up patterns. Repeated patterns are identified over time, allowing for more accurate results when more data is fed into the network.

For example, a network made to recognise traffic lights will start to find pixel patterns unique to them, from many traffic light image inputs. This is called **training** the network.

## 2. Sensor fusion

Whilst computer vision is very useful for the vehicle to make sense of the environment around it, image processing alone isn't enough all the time. This is where sensor fusion is needed. This involves the use of other sensors (e.g. LiDAR and Radar), along with the camera to build a complete understanding of the environment.



Furthermore, in certain weather conditions, LiDAR (Light Detection and Ranging) and Radar sensors may work better than cameras. They are also more accurate in getting distance data from the environment, allowing for more infallible calculations of velocity. Some autonomous vehicles make use of sensor fusion whilst others do not. For example, Tesla's Autopilot and Full Self-Driving (FSD) packages only use cameras and computer vision algorithms to determine what the environment looks like and how the vehicle should interact with it. Alphabet's Waymo, however, makes use of the complete sensor fusion of LiDAR, Radar, and cameras.

The data from these sensors are then combined using a 'filter'. One of the most popular filters is a Kalman filter, used to optimally predict things like the speed of another vehicle, which can't be measured directly. It can produce the 'best estimate' of the vehicle's surrounding environment. A Kalman Filter is an iterative mathematical process that uses a set of equations and takes consecutive data inputs, in order to quickly estimate the true value of the object being measured when the measured values have random errors, uncertainty or variation.



In the above example for temperature, the Kalman filter quickly narrows in on the actual temperature from a set of data points for temperature with a certain uncertainty.

This is the way in which 'noise' is removed from sensors in order to gain a more accurate value of what the sensor is measuring. Kalman filters get more complex when combining two or more variables together, such as in sensor fusion where data from cameras, LiDAR and Radar are combined to map the surrounding environment of the vehicle.

## 3. Localisation

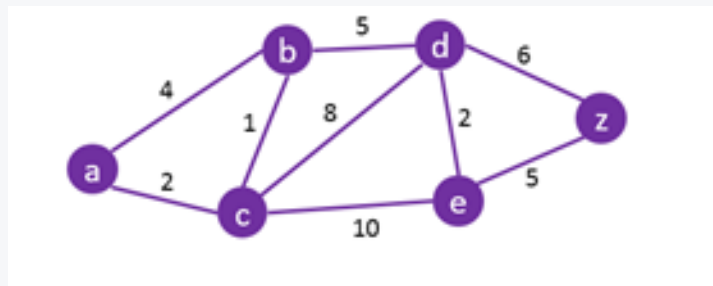
Localisation is the process by which the vehicle is located on a particular map, relative to its current location. It's assumed that this should be easy, with GPS technology to help find the location of the vehicle. Whilst GPS does allow the vehicle's location to be identified, it can only do so with an accuracy of 2m. The accuracy of 10cm is required to avoid a catastrophic situation in which the vehicle attempts to make a manoeuvre based on an incorrect assumption of where on the road it is. In this case, it's not feasible to depend on GPS technology.

Instead, sensor fusion can be used to collect distance data from the surrounding environment, which when combined with GPS using a filter, can enable the location of the vehicle to be more accurately pinpointed on a map.

#### 4. Path planning

Now that the vehicle can make sense of its surroundings and understand its exact location, it now needs to plan a route, or 'traverse' from point A to point B. This shouldn't be too hard as algorithms to do this have already been implemented and are used regularly on a large scale, e.g. when finding directions on google maps.

Directions are found most efficiently by using Dijkstra's algorithm. This algorithm solves a seemingly simple problem: What is the shortest path one can take to get from the starting point to the end? It is designed to find this shortest path in a weighted graph, where each edge between nodes is given a weight. This weight could represent the time taken, or distance between these nodes. It is a special case of the general A\* path-finding algorithm, which only focuses on reaching the end node, whereas Dijkstra's algorithm aims to find the shortest path possible.



In the graph above, the start node is a and the end node is z. The graph also contains weighted edges that connect two nodes:  $(n_1, n_2)$  denotes an edge, and  $w(n_1, n_2)$  denotes its weight. In the diagram shown above, the weight for each edge is written next to it.

The shortest path between them is found as follows:

1. Assign a distance value to every node by initialising an array  $\text{dist}$ , where  $\text{dist}(a) = 0$  and for all other nodes  $n$ ,  $\text{dist}(n) = \infty$ .

2. Initialise a priority queue  $Q$ , of all the nodes on the graph. This will be empty at the end of the algorithm.

3. Initialise  $S$ , an empty set of all the nodes on the graph that the algorithm has visited. At the end of the algorithm, this will contain every node on the graph.

4. Initialise  $\text{path}$ , an empty array that will contain the nodes present in the shortest path, in order, by the end of the algorithm.

5. While  $Q$  is not empty, pop the next node  $n_1$  (that is not already in  $S$ ) from  $Q$ , with the smallest  $\text{dist}(n)$ . (This node will be a in the first run of the algorithm, as  $\text{dist}(a)$  was initialised to 0)

6. Append the node  $n_1$  (that was popped from  $Q$ ) to  $S$ , indicating that  $n_1$  has been visited.

7. Update the  $\text{dist}$  values for each of the adjacent nodes  $n_2$  to  $n_1$ . Update  $\text{dist}(n_2)$  if  $\text{dist}(n_1) + w(n_1, n_2) < \text{dist}(n_2)$ , as in this case there is a new minimal distance found for  $n_2$ . If this is not true,  $\text{dist}(n_2)$  is left as is. If  $\text{dist}$  gets modified or  $n_1$  is the end node, then  $n_1$  is appended to  $\text{path}$ .

The algorithm visits all nodes in the graph and finds the smallest distance to each.  $\text{dist}(z)$  now contains the shortest path distance from a to z, and  $\text{path}$  would be  $[a, c, b, d, z]$

Path Planning also involves the vehicle having to calculate trajectories that it will follow over short distances, to help with tasks like turning on a curved road. This can be done by predicting where the other vehicles on the road are going to go, and then calculating what manoeuvre the vehicle should take in response, by building a series of waypoints to form a trajectory for the vehicle to follow.

## 5. Control

Control theory is the study of how force is applied to an object to control its movement. This is a crucial aspect of self-driving vehicles; if the vehicle isn't able to manoeuvre properly in the first place, mapping the vehicle's environment and finding the shortest route between two locations is useless. The vehicle calculates by how much it needs to accelerate and steer to follow the trajectory from path planning.

Control brings in an uncertainty to the process; it's not 100% accurate that the vehicle will follow the trajectory that it's meant to follow. It will, however, follow the trajectory very closely. This is also affected by conditions such as terrain and weather, which could cause fluctuations in the vehicle's speed and turning. It's also important to note that human drivers are not 100% accurate either; the path that a human-driven vehicle takes is not always a perfect trajectory through the middle of a lane.

### Levels of Automation

It is important to understand that there are various hurdles to overcome when trying to develop a 'fully autonomous' vehicle. Without overcoming these hurdles, it is essential that humans have some input to the system to ensure that the vehicle is safe, for both the passengers inside it and others in the surrounding environment (e.g. pedestrians). In order to gauge just how autonomous a vehicle really is (and how independent it is of humans), the Society of Automotive Engineers has created 6 levels of automation:

Level 0 automation is where there is very little input to the system; this is seen in most vehicles today, which are driven by humans. These vehicles may only have basic driver assistance features such as emergency braking.

Level 1 automation is more advanced and has functions that can drive the vehicle, but not fully.

These features usually focus on one aspect of driving e.g. acceleration (cruise control) whilst the driver monitors the other factors required for driving, such as steering the vehicle.

Level 2 automation is where both acceleration and steering are controlled by the vehicle and are referred to as Advanced Driver Assistance Systems (ADAS). This is akin to Tesla's Autopilot, which allows human override at any time. This human input factor means level 2 automation doesn't match the requirements to be classified as 'self-driving'.

Level 3 automation is the first type of automation where the environment would be monitored more by the system rather than a human. Level 3 vehicles have 'environmental detection' capabilities and can make informed decisions, but still require a human override.

Level 4 vehicles, however, can intervene by themselves in the case of a difficult situation, but still offer humans the option to manually override the system's decisions, meaning the vehicle is not fully self-driving.

The last level of automation, level 5 vehicles, will require absolutely no input from the human driver. This is what automakers are working towards. Level 5 autonomous vehicles will need to work like human drivers do, taking into account probabilities of certain events (such as a crash) happening.

Level 5 autonomous vehicles will need to work like human drivers do, taking into account the probabilities and uncertainties of certain events (such as a crash) happening, based on a number of factors. To do this, a vehicle may take into account the sensor data and predict whether a crash will happen or not.

There is a  $1/366 \approx 0.3\%$  chance that you will get into a car crash on a 1000 mile journey, which also means that you have a 99.7% chance of not getting into one. Let's say the vehicle has a test T that evaluates the chances of getting into a car crash based on real-time data from sensor fusion, and that this test has a 75% chance of correctly predicting a car crash. That also means it has a 15% chance of not predicting a car crash, also known as a false negative. Tests can also produce false positive results, where in this case T predicts that a car crash will happen even if it doesn't. This outcome could have a probability of 10%. That would also mean that the probability of it predicting 'no crash' correctly would be 90%. Since the situations 'crash' and 'no crash' have probabilities themselves, we can find the probabilities of the test producing each result:

'Crash': 0.3%

True Positive:  $0.003 \times 0.75 = 0.00225\%$

False Negative:  $0.003 \times 0.15 = 0.00045\%$

'No crash': 99.7%

True Negative:  $0.997 \times 0.9 = 89.73\%$

False Positive:  $0.997 \times 0.1 = 9.97\%$

So what would the probability of a car crash be if a positive test result is received from T? This is written as  $P(C|T)$ , which means the probability of C (The 'Crash' event) given that the test prediction has occurred. To find this, we use Bayes' Theorem, which states that  $P(C|T) = (P(T|C) \times P(C))/P(T)$ . So that would be  $P(C|T) = (0.75 \times 0.003)/(0.0997 + 0.00225) \approx 2.21\%$ , which is a pretty small chance of a crash happening and therefore indicates that there isn't much to be worried about. There's a catch, however: having such a high false positive rate brings down  $P(C|T)$  significantly; Assuming the test T was improved and the false positive becomes lower, e.g. 2%,  $P(C|T) = (0.75 \times 0.003)/(0.02 + 0.00225) \approx 10.1\%$ . Now that's a lot higher, and probably a cause for concern, indicating to the vehicle that it should try to make a manoeuvre to avoid this happening. This emphasises how important it is to have more accurate tests with lower false positives.

## The Trolley Problem:

Now that the various maths and algorithms used in self-driving vehicles have been explored, the scene for a famous thought experiment can be set: Imagine a self-driving car is boxed in on the road by a lorry, a motorcyclist and an SUV. Unexpectedly, a large and heavy object falls out of the lorry and is set on a collision course with the car. In this situation, there are only 3 decisions that the car can make: It can either choose to continue driving forward, resulting in casualties of the passengers inside. Alternatively, it could choose to avoid this by crashing into the motorcyclist or the SUV instead. Whilst a situation like this is highly unlikely to happen in real life, it is still important to consider: a human driver making a decision in that split second would not be judged for what they do; they had an instinctive reaction to the event. But a self-driving vehicle making a decision and causing casualties is a lot worse.

In order for an autonomous vehicle to be level 5, it must be able to handle all situations without any human interference. This includes the situation explained above. A programmer would cause the vehicle to perform a specific manoeuvre, in turn deciding who would die and who would live, making the whole situation look more like a homicide than an instinctive reaction. A self-driving algorithm may take into account that the SUV has a high safety rating and therefore crash into it, in order to minimise casualties in the scenario. Since the algorithm's choice would be influenced by the weights encoded into its probabilistic reasoning, any injuries and/or deaths from the scenario could mean that the programmer would be to blame. This would be a very confusing legal dilemma.

## Conclusion:

So, what does the future of autonomous vehicles have in store for us? Level 5 autonomous vehicles may still be a long way away, but what's most important is that we get it right, before we get the technology on our roads. There's also the potential for integrating swarm intelligence into autonomous vehicles, but that may take a little while longer to implement on a large scale.

Overall, we're going in the right direction, and this field will only continue to flourish in the future.

## Bibliography:

Eliot, L., 2018. Probabilistic Reasoning For AI Self-Driving Cars. [online] AI Trends. Available at: <<https://www.aitrends.com/ai-insider/probabilistic-reasoning-ai-self-driving-cars/>>

Synopsys.com. The 6 Levels Of Vehicle Autonomy Explained. [online] Available at: <<https://www.synopsys.com/automotive/autonomous-driving-levels.html>>

Griffin, D. What Are Your Chances Of Getting Into A Car Accident?. [online] News9.com. Available at: <<https://www.news9.com/story/5e6fca6cf86011d4820c3f2d/what-are-your-chances-of-getting-into-a-car-accident#:~:text=A%20report%20by%20Esurance%20found,collision%20once%20every%2017.9%20years>>

Brilliant.org. Dijkstra's Shortest Path Algorithm | Brilliant Math & Science Wiki. [online] Available at: <<https://brilliant.org/wiki/dijkstras-short-path-finder/>>

Babb, T. How A Kalman Filter Works, In Pictures | Bzarg. [online] Bzarg.com. Available at: <<https://www.bzarg.com/p/how-a-kalman-filter-works-in-pic>>

1Brown, 3Blue. But What Is A Neural Network?. Youtube. Available at: <<https://www.youtube.com/watch?v=aircAruvnKk>>

Raval, S. Self-Driving Cars Explained. [online] Youtube. Available at: <<https://www.youtube.com/watch?v=yt015gM-ync>>

Lin, P. The Ethical Dilemma Of Self-Driving Cars. [online] Youtube. Available at: <<https://www.youtube.com/watch?v=ixloDYVfKA0>>

Silver, D. How Self-Driving Cars Work. [online] Youtube. Available at: <<https://www.youtube.com/watch?v=Ly92UcnoEMY>>

The Economist. 2018. Self-Driving Cars Need Plenty Of Eyes On The Road. [online] Available at: <<https://www.economist.com/graphic-detail/2018/03/02/self-driving-cars-need-plenty-of-eyes-on-the-road>>

van Biezen, M. What Is A Kalman Filter?. [online] Youtube. Available at: <<https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.youtube.com%2Fwatch%3Fv%3DCaCcOwJPytQ&psig=AOvVaw1YkEKJBIVTG8ORMaQpA9ld&ust=1604652261167000&source=images&cd=vfe&ved=0CA0QjhxdFwoTCJCu2NOB6-wCFQAAAAAdAAAAABAD>>

Researchgate.net. 2020. [online] Available at: <[https://www.researchgate.net/profile/Knut\\_Kvaal/publication/239269767/figure/fig2/AS:643520205430784@1530438581076/An-illustration-of-the-signal-processing-in-a-sigmoid-function.png](https://www.researchgate.net/profile/Knut_Kvaal/publication/239269767/figure/fig2/AS:643520205430784@1530438581076/An-illustration-of-the-signal-processing-in-a-sigmoid-function.png)>

101 Computing. 2018. Dijkstra's Shortest Path Algorithm | 101 Computing. [online] Available at: <<https://www.101computing.net/wp/wp-content/uploads/Dijkstra-Algorithm.png>>

Reddit.com. 2020. How Do I Accurately Understand A Neural Network Diagram?. [online] Available at: <[https://www.reddit.com/r/learnmachinelearning/comments/9m561w/how\\_do\\_i\\_accurately\\_understand\\_a\\_neural\\_network/](https://www.reddit.com/r/learnmachinelearning/comments/9m561w/how_do_i_accurately_understand_a_neural_network/)>

# DEEP LEARNING - POTENTIAL TO TRANSFORM THE FUTURE

PRADHAY AMARNATH

## What is deep learning?

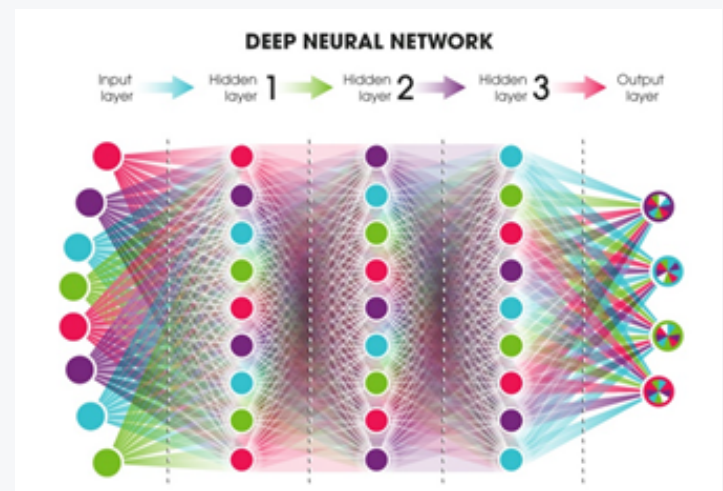
Deep learning can be considered as a subset of machine learning. It is a field that is fully based on learning and improving on its own by examining its computer algorithms. Compared to machine learning which makes use of simpler concepts, deep learning works with artificial neural networks; are designed to imitate how humans think and learn. Until recently, neural networks were limited by computing power and thus were limited in complexity. However, advancements in big data analytics have allowed us to build bigger, more sophisticated neural networks, allowing computers to observe, learn, and react to complex situations faster than humans. It can be used to solve any pattern recognition problem and without human intervention. Deep learning is a key technology behind driverless cars, enabling them to recognise a stop sign, or to distinguish a pedestrian from a lamppost. It is the key to voice control in consumer devices like phones, tablets, TVs, and hands-free speakers. Deep learning is getting lots of attention lately and for good reasons and it's achieving results that were not possible before.



## How does deep learning work?

Artificial neural networks architectures, comprising of many layers, drive deep learning, which is why deep learning models are often referred to as deep neural networks.

The term “deep” usually refers to the number of hidden layers in the neural network. Traditional neural networks only contain 2-3 hidden layers, while deep networks can have as many as 150. Deep Neural Networks (DNNs) are such types of networks where each layer can perform complex operations such as representation and abstraction that make sense of images, sound, text etc.



## What is a convolutional neural network (CNN)?

One of the most popular types of deep neural networks is known as convolutional neural networks (CNN). A CNN convolves learned features with input data, and uses 2D convolutional layers, making this architecture well suited to processing 2D data, such as images. Deep learning models are trained at first by using large sets of labelled data and neural network architectures that learn features directly from the data without the need for manual feature extraction, so you do not need to identify features used to classify images. The relevant features are not pretrained; they are learned while the network trains on a collection of images. This automated feature extraction makes deep learning models highly accurate for computer vision tasks such as object classification. CNNs used for deep learning learn to detect different features of an image using tens or hundreds of hidden layers.

Every hidden layer increases the complexity of the learned image features. For example, the first hidden layer could learn how to detect edges, and the last learns how to detect more complex shapes specifically catered to the shape of the object we are trying to recognize.

Neural networks are comprised of layers of nodes, much like the human brain is made up of neurons. Nodes within individual layers are connected to adjacent layers. The network is said to be deeper based on the number of layers it has. In an artificial neural network, signals travel between nodes and assign corresponding weights. A heavier weighted node will exert more effect on the next layer of nodes. The final layer compiles the weighted inputs to produce an output. Deep learning systems require powerful hardware because a large amount of data is being processed and involves several complex mathematical calculations. Even with such advanced hardware, some deep learning training computations can take weeks.

### How large does the dataset need to be and how does it train itself?

Deep learning systems require large amounts of data to return accurate results; accordingly, information is fed as huge data sets. When processing the data, artificial neural networks can classify data with the answers received from a series of binary true or false questions involving highly complex mathematical calculations. For example, a facial recognition program works by learning to detect and recognize edges and lines of faces, then more significant parts of the faces, and, finally, the overall representations of faces. Over time, the program trains itself, and the probability of correct answers increases. In this case, the facial recognition program will accurately identify faces with time.

### What is the difference between machine learning and deep learning?

Deep learning is a specialized form of machine learning. A machine learning workflow starts with relevant features being manually extracted from images.

The features are then used to create a model that categorizes the objects in the image. With a deep learning workflow, relevant features are automatically extracted from images. In addition, deep learning performs “end-to-end learning” – where a network is given raw data and a task to perform, such as classification, and it learns how to do this automatically.

A key advantage of deep learning networks is that they often continue to improve as the size of your data increases. In machine learning, you manually choose features and a classifier to sort images. With deep learning, the feature extraction and modelling steps are automatic.

A good illustration of this difference can be seen here:



### Deep learning in action:

Aside from your favourite music streaming service suggesting tunes you might enjoy, how else does deep learning impact our lives? As it turns out, deep learning is finding its way into applications of all sizes. With Facebook we may notice that it commonly identifies and tags your friends when you upload new photos. Digital assistants like Siri, Cortana, Alexa, and Google now use deep learning for natural language processing and speech recognition. Many email platforms have become adept at identifying spam messages before they even reach the inbox. PayPal has implemented deep learning to prevent fraudulent payments. Apps like CamFind allow users to take a picture of any object and, using mobile visual search technology, discover what the object is.

## Google and its work with deep learning



Google, in particular, has been leveraging the power of deep learning to deliver new and innovative solutions. For example, Google Deep mind's AlphaGo computer program recently defeated standing champions at the game of Go. DeepMind's Wave Net can generate speech mimicking human voice that sounds more natural than speech systems presently on the market. Google Translate is using deep learning and image recognition to translate voice and written languages. Google Planet can identify where any photo was taken. Google also developed the deep learning software database, TensorFlow, to help produce more A.I. applications.

### Final verdict

Deep learning is only in its infancy and, in the decades to come, will transform society. Self-driving cars are being tested worldwide; the complex layer of neural networks is being trained to determine objects to avoid, recognize traffic lights, and know when to adjust speed. Neural networks are becoming adept at forecasting everything from stock prices to the weather. Consider the value of digital assistants who can recommend when to sell shares or when to evacuate ahead of a hurricane. Deep learning applications will even save lives as they develop the ability to design evidence-based treatment plans for medical patients and help detect cancers early. These are just a few amongst all its other applications.

# AUTOPILOT SYSTEMS

RYAN THUKALIL

Even though the demand for travel abroad may have declined steeply in the current climate, we all associate a plane journey with shoving all the clothes in the suitcase the night before, slogging through the passport control lines that snake around the airport and worrying about how good the in-flight entertainment system is. However, very few of us ponder about the software, hardware and the pilots themselves playing a huge part in getting us to a new, exotic destination. After all, the pilots are almost always hidden from view by the door that separates the cockpit and the aircraft cabin, and even brief glimpses of the cockpit might seem entirely alien to us passengers. The closest we ever get to the pilots is when they speak through the intercom and inform us of the remaining flight time. Yet, while the pilots may be crucial in the navigation of the plane, it is the plane's autopilot which is usually running the show.



*Figure 1: A rare glimpse of the cockpit*

Essentially, the autopilot is a piece of software, and a part of the embedded, dedicated computer system that is the heart of the plane. While it is an example of automation, it is not an example of Machine Learning, which is a subset of AI, and involves studying computer algorithms in a bid to improve automatically through experience. While the autopilot may be smart, it is not smart enough and was rightly described as “dumb and dutiful” by Greg Zahornacky.

This is based on the concept that before each flight, the pilots have to prepare a flight route and input it into the control system, which is then used by the control system to maintain the flight's trajectory to a certain destination, as well as being logged into a database which can be used by the authorities to track the plane. The flight plan itself takes into consideration a variety of factors such as the flight duration, fuel, maximum take-off speed, weather conditions and many more.

Consequently, this eases the pressure on the pilots as they do not have to control the flight “hands on”, and reduces the likelihood of human error, since navigating a plane for hours on end with hundreds of passengers’ and staff’s lives literally in your hands can be tremendously stressful. On top of that, the autopilot gains insight into the conditions of the plane that the pilots simply would not have access to. The aspects of the plane’s computer equates to a relationship between the autopilot and the pilots, whereby the autopilot physically control the movement of the plane, while being closely monitored by the pilots, who can focus on other tasks such as communicating with traffic control and devising a contingency plan for adversary weather. While pilots are often stereotyped as “lazy” if they resort to autopilot in-flight, it could not be further from the truth as the autopilot system is incorporated into all modern airline systems, and pilots are actively encouraged to use it. Patrick Smith, an active commercial airline pilot, stated, “The auto flying system does not fly the airplane” rather the “pilots fly the plane through this automation”. The concept of the autopilot cannot be summed up any better than this!

The basis of the autopilot system is a negative feedback loop, and is not so different from the one in our body, which is used to regulate body temperature and other levels.

In a negative feedback loop, it will respond when the conditions of the system are not favourable, and will perform a certain action until the ideal conditions have returned. Each negative feedback system will be consisted of sensors, a control centre and effectors. In the same way, the autopilot system in a plane will take in data through sensors all over the plane, execute the instructions in the processor with its own fetch-execute cycle and adjust the plane accordingly. This system is critical in maintaining a certain condition such as velocity, altitude or direction, and an example of such would be if the wings, technically known as the ailerons, were no longer level. The sensors situated all over the plane which detect the imbalance would feed this information through to the processors, and after the computer determines something has to be done about this position imbalance, it would send a signals to the servos. The servo motor, a self-contained electrical device, is then able to move levers back and forth to adjust the wing surface of the ailerons, and this will continue to happen until the input from the sensors tells the processor that the ideal wing balance has been reached. This in turn forms a negative feedback loop, which is continually occurring several times per second, and is significantly smoother and quicker than what the pilot could achieve.

In the above example, the ailerons were adjusted to maintain an aircraft 's position and keep the wings perpendicular to the ground, and this is fundamentally what a single-axis autopilot would do. The objective of this type of autopilot is to control the ailerons alone, since any aircraft would rotate about its centre of gravity, so this autopilot is designed to keep the roll of the plane in check. The two-axis pilot can essentially be thought of as an upgrade to the single-axis autopilot, as it does everything the latter can do, but also incorporates the elevators into the system, which are the relatively small wings found at the back. This allows the computer to detect an unexpected change in altitude and point the nose up or down accordingly, consequently preventing the plane from stalling. The three-axis autopilot includes all of the features stated above, but additionally manages to move the rudder in order to change the direction in which the plane is heading in.

So what does this mean for the capabilities of the autopilot?

During the take-off and landing, there is a reason why passengers are advised to fasten their seatbelts (and squeeze their eyes shut), since there are the most dangerous phases of the journey. Although the autopilot may not be able to manage taxiing and take-off, powerful, innovative technology has meant the autopilot can guide and land the plane through threatening weather, such as fog, with minimal human intervention. This feature is known as Autoland which is available on major airport runways, and is tremendously helpful for the pilot. Surely, it is only a matter of time until we see the autopilot controlling the take-off as well...

When considering the autopilot software and the hardware that it is in control of, it is impossible to generalise the system for all aircraft, since they all have different purposes and would therefore need a unique dedicated system for doing so. However, the one feature they all share is that a real-time OS is implemented since it enables them to take in many inputs simultaneously, and respond immediately and appropriately to them. This is imperative in the running of the aircraft since the computer is continuously processing data and instructions from sensors all over the airplane, and if even one of the inputs are awry, the processors have to ensure the conditions return to normal, before it's too late.

As well as a suitable type of OS, the right processors are required, as the autopilot software needs to be able to take advantage of high processing power and performance. In the Boeing 777, triplicated FCP-2002 microprocessors are employed, since they are known to be highly reliable, making them suitable for constant commercial use. When designing the software, firms will often adopt a policy of design diversity, whereby each software module will be designed and implemented by different engineering teams, and each bit of software could potentially be run in entirely different architectures and languages.

While this becomes increasingly expensive as the amount of software and hardware needed increases, the concept of design diversity provides an ingenious safety feature because the whole plane will not rely on one engineering team alone, and so the same flaw will not occur in every bit of software. Therefore, if one component fails, there is a backup hardware to rely on, and this is a tremendous advantage of a real-time OS.

While both software and hardware are extensively tested to prioritise passenger safety, this does not mean the aviation industry is immune to heart-breaking incidents. In 2019, Ethiopian Airlines Boeing 737 Max crashed at Tulu Fara near Bishoftu, killing all 157 on board, and the second crash in five months involving the Boeing 737 Max. Although the blame was directed towards the Manoeuvring Characteristics Augmentation System (MCAS), which is quite different from the autopilot and used to balance the angle of flight by pushing the nose down, both systems rely on the many sensors situated on the plane. In this fateful accident, the sensor reading was incorrect and gave a false reading to the computer, leading to the plane being pushed down by the system, even though the pilots attempted to disable the MCAS many times. This highlights the vulnerabilities of using automation in such critical tasks, as all it took was a faulty sensor which led to the loss of numerous lives. With autopilot, which is also highly dependent on the sensors as well, it is clear to see that while it may reap many benefits, there is a high risk associated with it. Not only is it necessary that we ensure all technology is repeatedly tested, there has to be a clear relationship between the human pilot and the autopilot, the absence of which caused the Asiana Airlines crash in San Francisco quite a few years ago. Although there were significantly less casualties, namely 3, the sole cause of this crash was a misunderstanding of the purpose and capabilities of the automation since the pilots were apparently relying on an automatic throttle to maintain the speed. However, the plane was actually flying far too low and slow, resulting in the plane spinning and sliding erratically down the runway.



*Figure 2: The ill-fated Boeing 737 Max*

Although the current dip in demands for flights is certainly down to the pandemic and not automation errors, we must consider the limited capabilities of the autopilot, and it is essential that so do the pilots. While it may bring countless benefits to the aviation industry, it is vital that we keep in mind the fact that autopilot is clearly far from perfect. At the end of the day, we must remember that the autopilot is simply a computer software, and to correctly navigate the plane with hundreds of lives at stake, it must be flawless in both the manufacturing phase and in-flight when being supervised by the pilot. Has the autopilot technology really reached such a point?

### **Bibliography:**

Autopilot[online] Available at:  
<https://en.wikipedia.org/wiki/Autopilot>  
[Accessed 08/10/2020]

Cockpit Automation - Advantages and Safety challenges[online]Available at:  
[https://www.skybrary.aero/index.php/Cockpit\\_Automation\\_-\\_Advantages\\_and\\_Safety\\_Challenges](https://www.skybrary.aero/index.php/Cockpit_Automation_-_Advantages_and_Safety_Challenges)  
[Accessed 09/10/2020]

Andrew J.Hawkins Deadly Boeing crash raises questions about automation[online]Available at:  
<https://www.theverge.com/2019/3/15/18267365/boeing-737-max-8-crash-autopilot-automation>  
[Accessed 10/10/20]

# ALGORITHMIC BIAS

ADVAIT MENON

Bias is defined as “a disproportionate weight in favour of or against an idea or thing”. In the context of algorithms, an algorithmic bias is “a systematic, repeatable error that can create unfair outcomes”. So how does this bias in algorithms manifest and what can be done about it? Humans can be biased towards a range of different ideas such as race, gender and religion, algorithms are no different. The use of computer systems in practically every industry today creates the potential for algorithmic bias to be present anywhere.

Search engines such as Google and Bing use a concept called ‘word embeddings’. Through this method, words are represented as vectors in many dimensions, and related words are placed close to each other. A human can easily answer the analogy “man is to woman as king is to”, but this proves difficult for an algorithm with no conceptual understanding of a language. Using word embedding (Figure 1), a computer could calculate the answer as follows:  $\text{king} + (\text{woman} - \text{man}) = \text{queen}$  (notice how the vector from man to woman is the same as from king to queen).

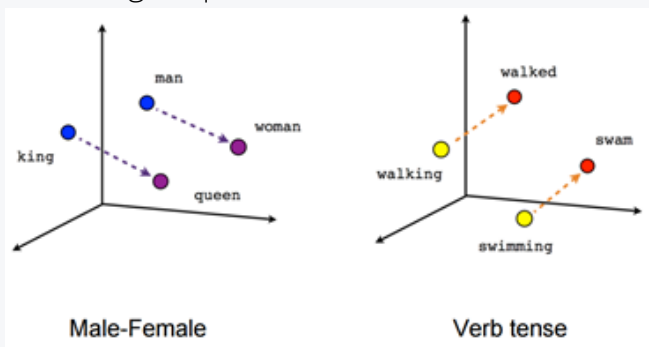


Figure 1: word embedding

While this is an extremely clever technique, many studies have shown that these word embeddings are prone to gender biases. One study of the word embedding algorithm ‘GloVe’ showed that believed the word “he” was related closely related to occupations such as “philosopher”, “captain”, “architect”, “warrior”, “boss”, whereas “she” was more closely related to “homemaker”, “nurse”, “receptionist”, “hairdresser” and “nanny” (see Figure 2).

Similar biases have also been found in ‘word2vec’ – the word embedding algorithm used by Google. One method of debiasing the algorithm is to calculate the “direction” of the bias (e.g. is it closer to “she” or “he”) for each word and then translate it so that it is equally spaced from male and female terms. For example, moving “homemaker” so that it is equidistant from “she” and “he”. There is a noticeable difference after debiasing which can be observed in Figure 3. But why is GloVe biased? GloVe utilises unsupervised learning – which means it draws its own conclusions from the data it trains on. The algorithm shown in Figures 2 and 3 was trained on words appearing in a wiki and established its own understanding of the relationships between them; so the algorithm independently reproduced the biases appearing in the training data.

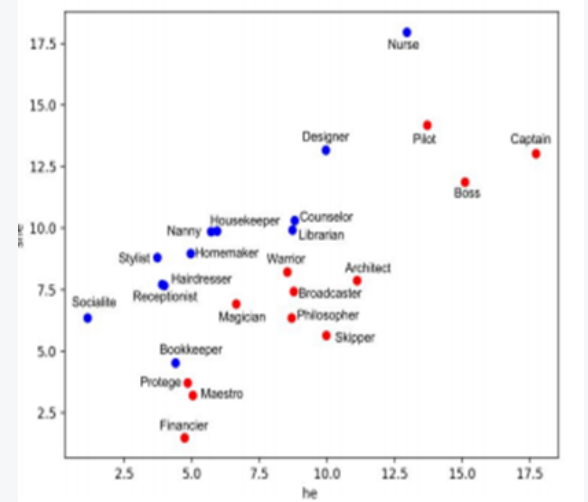


Figure 2: red dots closer to “he”, blue dots closer to “she”

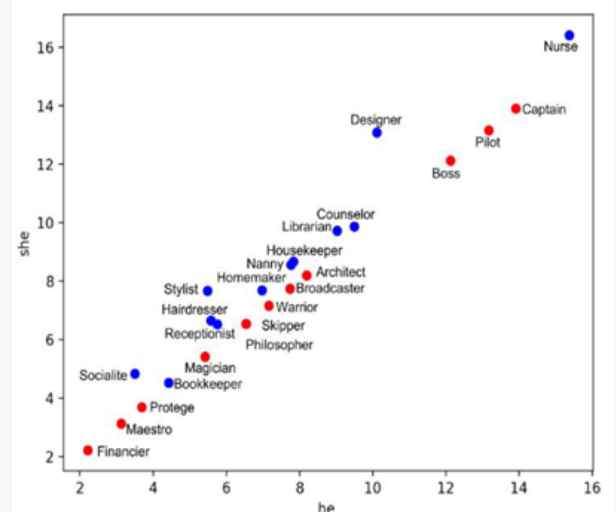


Figure 3

Some algorithms can have a significant impact on people's livelihoods, as such, it is important that they are not biased against any groups of people. COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) is an algorithm used in the American judicial system to predict the 'risk' of a defendant (in other words, a high risk defendant is someone who will be likely to reoffend). In a study, the newsroom ProPublica found that COMPAS was biased against African Americans – with 45% scoring a higher risk than they actually were. Comparatively, the study found that the error level for white defendants was only 23%. This is a false positive – i.e., a person who is not going to commit a future crime is assigned a high risk. It is the worst possible outcome for a defendant because it means they could be serving extra time when in fact they won't reoffend. The study also showed that white defendants experienced more false negatives i.e., the algorithm classified a reoffending defendant as low risk.

Despite COMPAS seeming terribly biased from these statistics, the creators (Northpointe) argued that it wasn't since there was no calibration bias. This means that for both races, similar proportions (around 60%) of those classified as high risk did reoffend. Neither ProPublica nor Northpointe are wrong in their assessments, but it brings to attention the problem of defining bias. From Northpointe's point of view, COMPAS is unbiased as there is no calibration bias – but this means that different races will get different false positive and negative rates. From ProPublica's point of view, COMPAS would be unbiased if the false positive and negative rates for each race were equal – however this would inevitably lead to a calibration bias. Either way there will always be some form of bias.

So how can bias be eliminated?

The roots of algorithmic bias lead back to the problems in our present-day society. Algorithms are extremely good at spotting patterns, so when they train on biased data, they learn to reflect the same biases they discover. In the ideal world, society as a whole would be unbiased – instantly solving the problem of algorithmic bias along with it, but this is very unrealistic.

However, we can reduce algorithmic bias drastically by controlling the data available to the algorithms. By ensuring that the training data is unbiased the algorithm won't end up learning biases as there aren't any for it to learn. For example, a facial recognition AI might train on a dataset containing equal proportions of faces from all races and genders so that it is equally good at identifying them all. It is also key to note that in some cases, bias may not always be a bad thing. For example, a text generation AI writing in the style of Jane Austen may want to pick up the biases in her novels to better mimic her.

To conclude, algorithmic bias today will have different definitions depending on who you ask and is extremely difficult to quantify. It will never truly be eliminated until we have first eliminated the bias in our own society, but the best we can do at the moment is mitigate its negative impact and ensure all algorithms are trained to be as fair as possible to everyone.

## **Bibliography:**

Authors, V., n.d. Algorithmic Bias. [Online]  
Available at:  
[https://en.wikipedia.org/wiki/Algorithmic\\_bias](https://en.wikipedia.org/wiki/Algorithmic_bias)  
[Accessed 11 12 2020].

Authors, V., n.d. Wikipedia. [Online]  
Available at: <https://en.wikipedia.org/wiki/Bias>  
[Accessed 11 12 2020].

CVera, M. F. L., n.d. Stanford University. [Online]  
Available at:  
<https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1184/reports/6835575.pdf>  
[Accessed 11 12 2020].

Khandelwal, R., 2019. Word Embeddings for NLP. [Online]  
Available at:  
<https://towardsdatascience.com/word-embeddings-for-nlp-5b72991e01d4>  
[Accessed 11 12 2020].

Sumpter, D., 2018. Outnumbered. Illustrated Edition ed. s.l.:Bloomsbury Sigma.

# 5G: IS IT REALLY GOING TO KILL US ALL?

MIN JIE GUO

PUBLISHED IN THE ACADEMIC JOURNAL

Before we delve into the many wild theories speculated about 5G, what is it and why is it such a big deal? 5G is the fifth generation of mobile internet connection, offering significantly higher Gbps peak data speeds. It is a new global wireless standard after 1G, 2G, 3G, and 4G networks. Due to extended use of the radio spectrum, it enables a new kind of network allowing more devices to access the mobile internet simultaneously. Imagine doing everything you do now, but up to 100x faster. Picture not having to wait what seems like eons to download movies on Netflix using your mobile data - you could download a two-hour movie in just 3.6 seconds! This wireless technology seemingly offers a plethora of advantages - almost sounding too good to be true.

Remote actions could be performed in real time due to a lower latency. Health trackers could alert emergency services the instance something wrong is detected, and doctors could precisely intervene in surgeries done by remote manipulators (telem manipulators) on patients on the other side of the world. There are military implications; it can be weaponised - missiles could be controlled with utmost precision, as an example. This is dangerous, but technically, this can all already be done by 4G and 3G.

Technology can be further advanced. Smart cities can be developed due to greater network capacities and the autonomous vehicle could reach new heights (sensors could be placed all over the city, which could analyse traffic and choose more efficient routes, reduce the number of accidents by reducing blind spots, find available parking spaces, etc.). All connected devices will have instant connections to the Internet, which will be able to exchange information with each other in real time, favouring the Internet of Things. Drones would be controlled with a much higher precision and able to send back a high-definition video almost instantaneously.

There would also be a huge boost for VR and AR (Virtual/Augmented Reality) apps, which then themselves have a multitude of applications in a wide variety of industries as the technology develops, like interacting with CAD models in engineering and industrial design.

5G also supports network slicing. Subnetworks allow specific characteristics to be given to parts of the network, optimising connectivity, speed, and capacity to suit diverse requirements of different applications.

Naturally, there are disadvantages. An increased bandwidth will mean that there is less coverage, so more cell towers will be required. This will mean more visual pollution and that as towers are built (and not burnt down), users will not have as widespread a coverage at first. A new phone would also have to be purchased as a 5G chipset and modem would be required, which would be expensive and could lead to a lot of E-waste due to irresponsible disposal of old devices.

There are also many security and privacy issues that need to be resolved. Currently, there are many vulnerabilities that could allow denial of service (DoS) attacks and the hacking of public channels, like emergency response and traffic control. If hacked, it could be disastrous. The network can be vulnerable to security holes when switching between 5G, 4G and 3G that were unaddressed in the previous generation. Many also fear the manufacturer will engineer a 'backdoor' allowing mass data surveillance. This circulates to claims made against network infrastructure provider Huawei, most famously by Donald Trump, and lead to racist backlash as these claims seemed to be more politically concerned than concerned around cybersecurity. Security and intelligence experts in the UK assert that any risk to national security can be mitigated as the infrastructure providers' role can be kept to non-core parts of the network.

As 5G is being implemented worldwide, articles are being rolled out left, right, and centre. What is fact and what is fiction?



*Figure 1: A January 2020 protest against 5G rollouts at London's Parliament square, taken by Bradley Stearn/Shutterstock*

Anti-5G campaigners claim that it increases the risk of cancer, causes internal damage to our cells and even neurological damage. However, these are the same claims that were made when 2G, 3G, 4G and Wi-Fi were launched. Over-exposure to any man-made technologies could potentially be harmful, but 5G is no less safe in comparison to its predecessors. I have never read an article saying that Wi-Fi was going to kill me, but I have read an awful lot recently about 5G. (If anything, people say they would die without Wi-Fi.)

It is claimed to have an extreme, adverse effect on human health and lead to the degradation of our immune systems. 5G is transmitted over radio waves, which are part of the electromagnetic spectrum and it is true that higher frequency ionising radiation at the end of this spectrum does pose dangers. Although it is at a slightly higher frequency than previous networks, it is still in a band of low-frequency waves that are non-ionising. Overwhelming scientific evidence shows that this non-ionising radiation causes no internal damage to our cells.

UK communications regulator, Ofcom, recently published the results of its latest spectrum measurement tests, tracking EMF (Electromagnetic Field) emissions near 22 of the busiest 5G mobile phone base stations. It found that the maximum emissions measured at any of the test sites was 1.5 per cent of the levels set as guidelines by the International Commission on Non-Ionising Radiation Protection (ICNIRP). [1]

Public Health England states that 'the overall exposure is expected to remain low relative to guidelines and, as such, there should be no consequences for public health.' [2] The ICNIRP guidelines go up to 300GHz, whereas the maximum for 5G will likely only be in the tens of GHz. Going out in the sun means you are exposed to electromagnetic radiation far greater and more ionising than what is emitted by 5G cell towers. But there are no protests (that I know of) against the sun.

However, just because there is no 'convincing' evidence that non-ionising radiation has a biological effect does not mean that it is completely safe or that no effect exists. Researchers continue to conduct studies out of fear that prolonged exposure has adverse effects. A widely quoted study about cell phone radio frequency radiation found that high exposure to 3G radio-frequency radiation (RFR) led to some cases of cancerous heart and brain tumours, and tumours in the adrenal glands of male rats. It has been pointed out that the number of tumours was statistically so small that they could have occurred by chance, which is likely as they were only detected in male subjects, and that the level and duration of exposure was significantly greater than what any human would ever be exposed to. Additionally, the irradiated rats lived longer than the unexposed control rats. This study is therefore seen by many researchers as insignificant and considered to have proven no real data. [3] As this study shows, research into radiation risks is difficult and often inconclusive, meaning it can take a long time to make real progress.

While the ICNIRP has maintained that there is no evidence that there are no adverse biological effects of RFR except tissue heating at exposures above prescribed thresholds, RFR has been classified as Group 2B, a possible human carcinogen, but so are pickles and coffee. The World Health Organisation launched a draft of a monograph on RF fields, but this has been dismissed as 'suggestive, but unconvincing' evidence. The only consistently recognised effect in humans is heating. [4] More thorough research needs to be conducted to prevent potential health risks.

Despite the lack of consensus that 5G will kill us all, 5G-protective equipment is currently trending in the markets. A 'must-have' is the highly recommended £339.50 5GBioShield USB Key, despite an estimated manufacturing cost of £5, with a 'quantum holographic nano-layer catalyser, said to 'balance and re-harmonise the disturbing frequencies.' It has been proven by cyber-security experts and London Trading Standards as no more than a basic USB drive. [5][6] Many of these products make use of scare tactics for marketing and later of the placebo effect. None of these products are considered to be backed with thorough research.

The COVID-19 outbreak has also led to several conspiracy theories, namely that 5G has either caused, helped or accelerated the spread of the virus and that instead of facemasks guarding against the virus, we should all be wearing PPE to guard against 5G waves. There are even claims that the virus does not actually exist and that it is just a cover-up for the real problem: 5G waves (it is quite clear that the virus very much does exist and the virus does also exist in countries without 5G).



*Figure 2: £20 pound note with alleged 5G tower and coronavirus symbol, Craig Lewis on Facebook*

The symbols on the new £20 note are said to reference a 5G tower giving off radiation and its link to the coronavirus. The tower is the Margate lighthouse, the radiation is Margate's Turner Contemporary art gallery and the coronavirus symbol is a purple foil security patch based on the design of a staircase at the Tate Britain.

Other theories suggesting that the government was cutting down trees to hide the effects 5G has on them was also disproved as the circulated picture was of felled trees in Serbia as part of redevelopment works to be replaced with new trees. 5G was also cited as the reason behind the death of 297 birds in the Netherlands, but it was not being tested in the vicinity of the park at that time, confirmed by the Dutch government [7] , and mass starling death is not unheard of. Tests designed showing the effects of cell phone generated EM waves on honeybee navigation were also dismissed due to small sample sizes and the fact that tests did not involve cell phones. There is yet to be scientific confirmation of any theories.

It has however been confirmed that COVID-19 is transmitted via respiratory droplets and just to be clear, droplets cannot be transmitted through 5G waves. Despite this, phone masts have been set alight across Europe, threatening to cut off vital communications when we are all relying on connectivity more than ever. Telecoms engineers have even been threatened and stabbed by enraged protestors, despite them just doing their jobs.

To re-emphasise: for now, there is no concrete evidence that there are health risks and the scientific consensus is that it is safe.

## References:

- 1) Ofcom. (2020) Electromagnetic Field (EMF) measurements near 5G mobile phone base stations. [online] Available at: [https://www.ofcom.org.uk/\\_\\_data/assets/pdf\\_file/0015/190005/emf-test-summary.pdf](https://www.ofcom.org.uk/__data/assets/pdf_file/0015/190005/emf-test-summary.pdf) [Accessed 31. May 2020]
- 2) Public Health England. (2019) Guidance. 5G technologies: radio waves and health. [online] Available at: <https://www.gov.uk/government/publications/5g-technologies-radio-waves-and-health/5g-technologies-radio-waves-and-health> [Accessed 31. May 2020]

3) National Toxicology Program, U.S. Department of Health and Human Services. (2018) Cell Phone Radio Frequency Radiation. [online] Available at: <https://ntp.niehs.nih.gov/whatwestudy/topics/cellphones/index.html> [Accessed 31. May 2020]

4) National Cancer Institute. (2020) Cell Phones and Cancer Risk. [online] Available at: <https://www.cancer.gov/about-cancer/causes-prevention/risk/radiation/cell-phones-fact-sheet#what-is-radiofrequency-radiation-and-how-does-it-affect-the-human-body> [Accessed 1. June 2020]

4) Cellan-Jones, R. BBC. (2020) Trading Standards squad targets anti-5G USB stick. [online] Available at: <https://www.bbc.co.uk/news/technology-52810220> [Accessed 31. May 2020]

5) Davies, J. (2020) The con artists profiting from 5G conspiracy theories. [online] Available at: <https://telecoms.com/504628/the-con-artists-profiting-from-5g-conspiracy-theories/> [Accessed 31. May 2020]

6) Antennebureau. (2018) Geen 5G-testen bij Huijgenspark in Den Haag. [online] Available at: <https://www.antennebureau.nl/actueel/nieuws/2018/november/14/geen-5g-testen-bij-huygenspark-in-den-haag> [Accessed 31. May 2020]

# FRACTALS IN PROGRAMMING

JANET THOMAS ALBERT

The word fractal was derived from the Latin word fractus, meaning broken. They exist both in reality and abstractly; the latter is a set of complex pictures generated through a computer iterating a single formula. Recursive in nature, they are infinitely complex patterns that are self-similar across different scales. The utilisation of fractals in programming range from analysing galaxies, data compression, depicting bacteria, chemical reactions, human anatomy and diffusion to fractal art, music, and special effects.

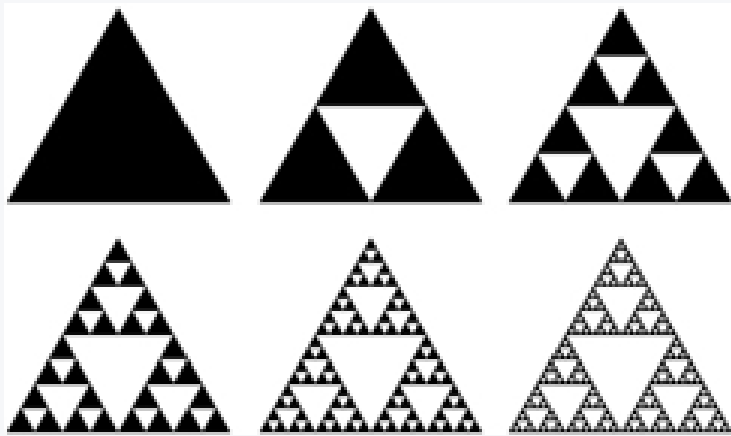


Figure 1: Sierpinski triangle – example of a fractal

The ideology of generating fractals can be explored through simpler geometric shapes. Starting with a shape called the initiator, subparts of which are then replaced with a pattern to produce the generator. As Figure 2 shows, each straight-line segment in the initiator is replaced with four equal length line segments at each step, the scaling factor is  $1/3$ . Additional to this, the length of each line segment in the initiator increases by a factor of  $4/3$ . As a result, the length of the fractal curve tends to infinity as increasing detail is added to each curve.

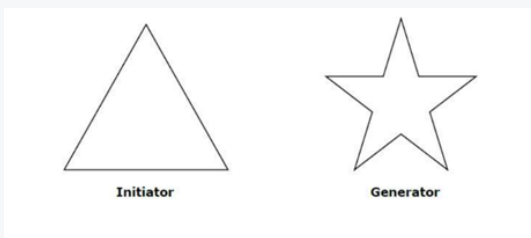


Figure 2: example initiator and generator

| Segment Length = 1 | Segment Length = $1/3$ | Segment Length = $1/9$ |
|--------------------|------------------------|------------------------|
|                    |                        |                        |
| Length = 1         | Length = $4/3$         | Length = $16/9$        |

Figure 3: progression of steps

Fractal compression is a type of lossy compression method used for digital images, the use case being a texture or natural image due to the fact many parts of these images often resemble each other.

Fractal algorithms are utilized to convert these repetitions within the image into mathematical data called “fractal codes”, these are then used to recreate the encoded image. Fractal image representation is synonymous mathematically as an iterated function system (IFS). In the case of application to a binary image, an image consisting of pixels of exactly two colours, typically black and white. The image can be thought of as a subset of  $R^2$ . A real coordinate space of dimension  $n$  written  $R^n$ , is the coordinate space over the real numbers meaning that  $n$  – tuples (sequences) of real numbers. Due to the characteristics of component wise addition and scalar multiplication it can be defined as a real vector space. Therefore,  $R^2$  can be viewed as a two-dimensional space. Figure 4 shows a binary image composed of only black and white pixels on a two-dimensional plane.

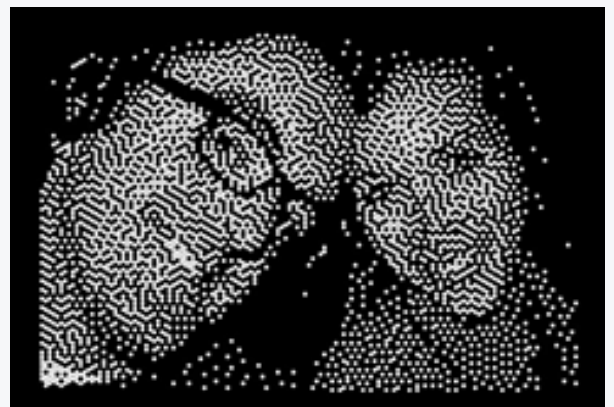


Figure 4: a binary image

The IFS is a set of contraction mappings  $f_1 \dots f_n$ . The definition of a contraction mapping is, supposing  $(X, d)$  is a metric space.  $X$  being a set and  $d$  being a distance function. A function  $T: X \rightarrow X$  (that maps  $X$  onto itself), is said to be a pseudo-contraction if for any two points  $(x, y)$  in the set  $X$  the distance between the two points  $x$  and  $y$  are strictly greater than the distance between their images. This can be written as such:

$$d(T(x), T(y)) < d(x, y)$$

Equation 1: pseudo-contraction condition

But they are a contraction if there exists a real number  $\lambda$  strictly between 0 and 1 such that the distance between the images of the points  $x$  and  $y$  is less than or equal to  $\lambda$  multiplied by the distance between  $x$  and  $y$ . This can be denoted as:

$$d(T(x), T(y)) \leq \lambda d(x, y)$$

Equation 2: contraction condition

Since a contraction requires a more restrictive condition to be satisfied, all contractions are pseudo contractions. Since the set is self similar, IFS is a fractal representation of the set. Likewise, IFS representation can also be extended to a greyscale image by considering the image's graph as a subset of  $R^3$ , three-dimensional space. Similarly, to the binary case, the set is described by an IFS using a set of contraction mappings but in  $R^3$ .

Features of fractal compression revolve around the fact that encoding is very computationally expensive due to the search that is used to find self similarities. Contrast to this, decoding is relatively fast. Despite, asymmetry so far making it impractical to use in real life applications when video is archived for distribution from disk storage or conversely file downloads, fractal compression gains a significant competitive edge.

An intrinsic feature of fractal compression is that images are resolution independent after conversion to fractal code occurs.

This occurs since the IFS in the compressed files are infinitely scaled (fractal scaling) meaning the resolution independence of a fractal encoded image can be used to increase the display resolution of the image (fractal interpolation). In fractal interpolation, the image is encoded into fractal codes via fractal compression and then decompressed at a higher resolution resulting in an up-sampled image wherein the IFS has been used as the interpolant.

The Mandelbrot set is an abstract set of fractals, the first computation of which was done by Pierre Fatou. It is named after the mathematician Benoit Mandelbrot due to his pioneering work in the field of fractal geometry. The computation of this set of numbers has provided breakthroughs in the field of mathematical visualisation. The Mandelbrot set is plotted within the complex plane meaning that  $a+bi$  is such that  $a$  and  $b$  are real numbers whilst  $i^2=-1$ . It is interesting to consider the distance of numbers in the form  $a+bi$  to the centre point denoted as the magnitude. This magnitude holds inherent mathematical qualities of significance to us.

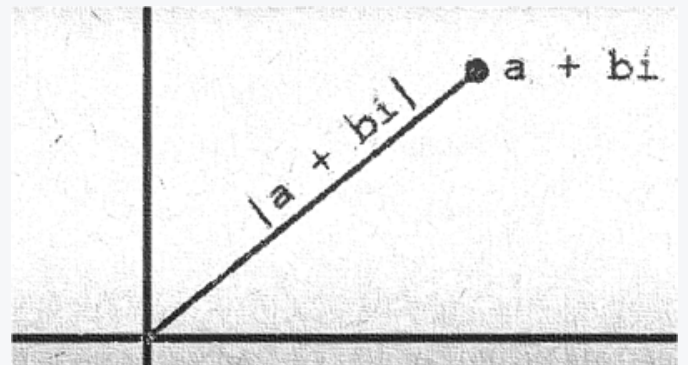


Figure 5: magnitude of  $a + bi$

If we take complex number  $c$  and associate it to the function  $f_C(z)=z^2+c$ ,  $z$  being some random value. Investigating the behaviour of zero, when  $c$  is 1, under the function would produce the following results:

|                    |      |
|--------------------|------|
| $f_1(0) = 0^2 + 1$ | : 1  |
| $f_1(1) = 1^2 + 1$ | : 2  |
| $f_1(2) = 2^2 + 1$ | : 5  |
| $f_1(5) = 5^2 + 1$ | : 26 |

When we consider the effect that the function has on the magnitude (distance from zero), 2 cases can be observed for the function  $f_C(z)=z^2+c$  . Firstly, the distance from zero of the sequence produced gets arbitrarily large. Secondly, the size can get bounded (never gets larger than 2!, in this case). As per the first investigation, regarding the behaviour of zero when c is one the output follows the first option wherein it converges to infinity. However, if we apply the same behaviour of zero changing the value of c to -1 we witness the following:

$$f_{-1}(0) = 0^2 + -1 : -1$$

$$f_{-1}(-1) = -1^2 + -1 : 0$$

$$f_{-1}(0) = 0^2 + -1 : -1$$

And so, the iterates simply alternate between the values of minus one and zero, highlighting an example of case two wherein the size of the sequences s bounded in on itself. As a result, one definition of the Mandelbrot set is the set of complex numbers c which case two holds. If we plot such numbers into the complex plane, we get the Mandelbrot set. All numbers outside the set comply with case two. As the value of c moves around it is not possible to predict the value that the function will take or the case that applies to it.

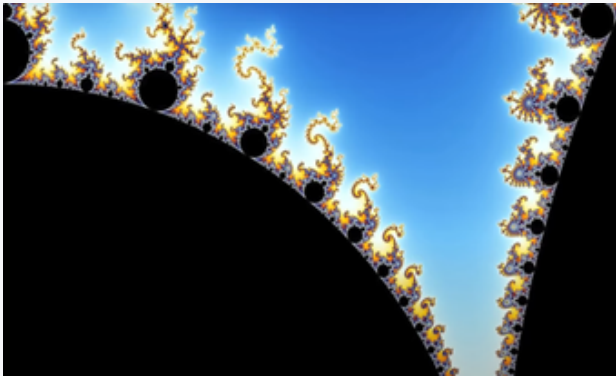


Figure 6: edge of Mandelbrot set

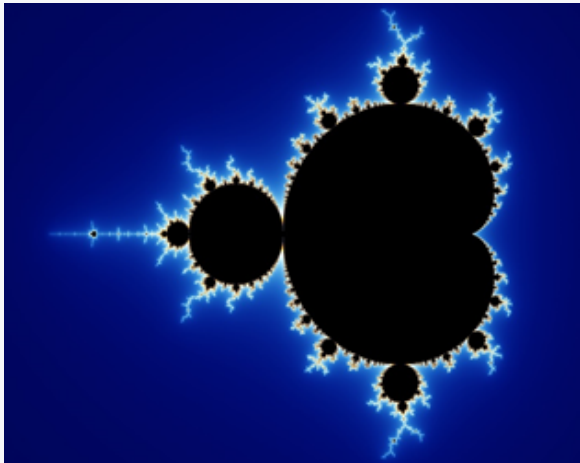


Figure 7: the Mandelbrot set

Fractal landscapes are surfaces that are generated through the use of a stochastic algorithm that is created to produce fractal behaviour that mimics the appearance of natural terrain. It can also be said that the output is a result of a procedure that is not a deterministic fractal surface rather that of a randomly generated surface that exhibits fractal behaviour. Perlin noise can also be considered as a simple fractal and is used widespread in the field of computer graphics. It is a type of gradient noise which is a procedural texture primitive that is used to increase the appearance of realism in computer graphics by visual effect artists. The function boasts a pseudo random appearance, yet all its visual details are of equal size which provides the property that it is readily controllable.

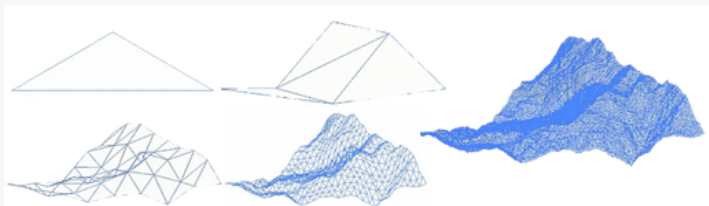


Figure 8: rendering a fractal mountain

We are surrounded by fractals in everyday life and so their application to programming could be considered to be a feat of human creativity. As Benoit Mandelbrot puts it himself “fractal geometry is not just a chapter of mathematics, but one that helps Everyman to see the same world differently.”

Enigma is undoubtedly the most famous cyphering machine on earth. Originally designed by German inventor Arthur Scherbius, the machine was used to encrypt messages prior to radio transmissions (and decrypt on reception), most notably by the German armed forces during WW2. Polish codebreakers could read messages sent by Enigma as early as 1933, but their technique exploited a German procedural error of repeating the machine's settings. The settings of an Enigma machine are everything – a message cannot be decrypted simply by having another Enigma machine, it must also be set up correctly. The aim of the codebreakers was to deduce these settings, as a unique one would be used for all messages sent each day. In the British codebreaking facility at Bletchley Park, Sir Alan Turing set about to design a machine which could break Enigma without any reliance on the German's system of repeating certain settings, which they were likely to change once they realised its weakness. Sure enough, in May 1940 the Germans did switch to a new procedure, and by August that year Turing's machine was ready. It was named Bombe, a Polish expression meaning 'Eureka', and is truly a work of algorithmic and computational genius.

The German operator would press down a key (for the letter they wished to send), giving an input by completing an electrical circuit, and a letter on the display would light up, giving the encrypted character. The main method of encryption was passing this current through three sequential rotors, each of which would switch every letter onto a different one. The current would then be switched onto yet another letter (in the stationary Umkehrwalze, see figure 2), before being run back through the same rotors for added encryption. With every press of a key, the first rotor would rotate one place, so each letter would then switch onto a completely different one. With every complete rotation of the first rotor, the second would move one place, and with every complete rotation of the second the third would also move one place. This was what made Enigma so hard to crack – the encryption process was different for each and every letter typed. The input 'A' could produce an output of any other letter within the same message without even changing any of the machine's settings! Every other letter that is, except itself. As the Enigma machine also had to be used for decrypting messages, everything was mirrored, meaning if 'A' encrypts to 'B' then 'B' encrypts to 'A' (and so we have the decrypted letter). However, this meant that no letter could be encrypted as itself – as the message was passed back through the same rotors, to encrypt as itself the stationary 'swap' when the current turned around a letter would have to always map onto itself. As it would not be very hard to break a code where the letter 'A' is always encrypted as 'A', the machine was instead designed so that this could never happen. And that is what allowed the code to be cracked. [1]



Figure 1: Enigma – three rotors at the top, output lights, input keys and the Stecker

To understand how the Bombe cracked the Enigma, the Enigma machine itself must first be understood. Its abstracted operation uses a familiar function of modern computing: input – process – output.

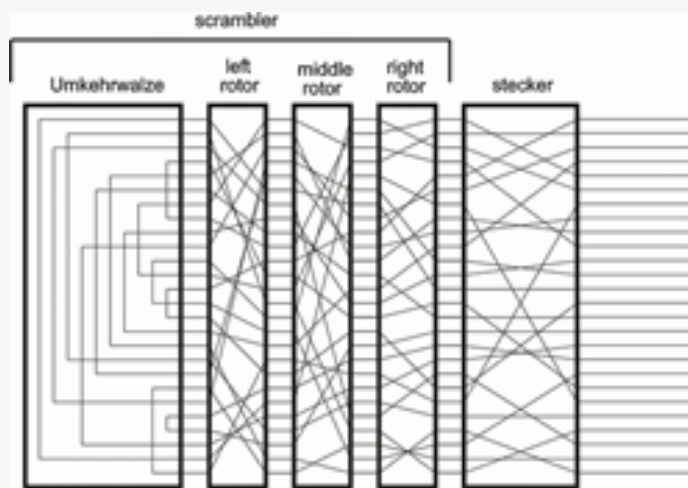


Figure 2: an electrical diagram of the Enigma machine, with the input and output channel for each letter on the right

Standard trial and error codebreaking methods just did not work with Enigma – there were far too many possibilities to be tested. For a start, the three main rotors were selected from five potential switching options, could be arranged in any order and set to any ‘rotated’ position to start with. As an additional layer of complexity, each input and output was run through a switchboard (Stecker) before or after processing respectively, which gave the operator the option to map any pair of letters onto one another. This meant that an ‘A’ could in fact be processed as if it were a ‘Y’, and the output ‘X’ displayed as ‘B’ (and vice versa for decryption purposes) – up to a total of 10 letter pairs. If you run the maths, there are more than  $1.07 \times 10^{23}$  possible starting settings of Enigma, one of which would be randomly selected daily and contained in every operator’s codebook. The challenge faced by those at Bletchley Park was to figure out precisely what this setting was, by examining intercepted messages, within the timespan of that day. To brute force every combination would take the current population of earth, at one attempt a second, 435 thousand years. Which is slightly more than a day, and uses slightly more people than were available. There had to be a better way.

The Bombe machine was developed for that very purpose, but even a machine couldn’t simply brute force so many combinations. The problem with this kind of approach is that the 6-letter cyphertext ‘HDOWBG’ could just as well be ‘OLAVES’ as ‘ATTACK’ – how can you tell when you have the correct answer?

An additional issue with Enigma is that many settings could produce the same 8 letter result, but the rest of the message could be completely different. A comprehensive solution was needed, and that started by cheating. To begin to know whether deciphered text was correct, a crib was first created, mapping a short section of known cyphertext onto its associated plain text – the word ‘Wettervorhersage’ (meaning weather forecast) was often used near the beginning of messages to convey that information, so the codebreakers knew it would be in there somewhere. Next, the codebreakers could take advantage of the fact that no letter ever encrypts as itself, in order to align the plaintext with the cyphertext for that word (if ‘A’ mapped to ‘A’, then that word was not matched to that cyphertext – note that this was trial and error!). Once the crib had been found, a menu could be deduced. This was a diagram which showed the relation between pairs of letters, and at what relative position in the sequence of rotating rotors that pair would be generated. At this point, the codebreakers had all they needed to program the Bombe.



Figure 3: an example crib

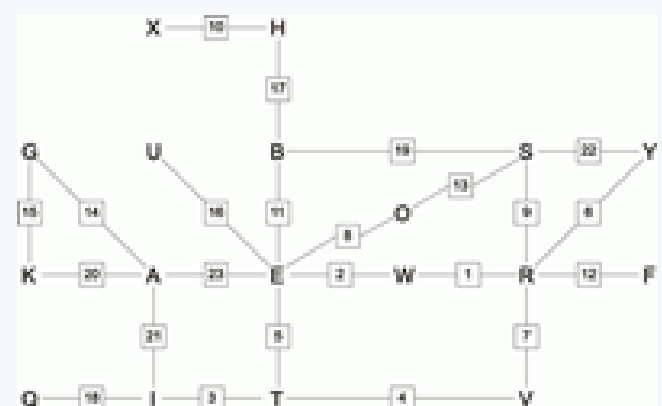


Figure 4: the corresponding menu

At its most basic, the Bombe is a machine which checks for logical contradictions in the potential settings of the Enigma machine. The Bombe itself contains sets of three rotors, just like the Enigma, which could also be interchanged to any of the 5 the Enigma uses – yet another guessing game for the codebreakers.

All the sets of 3 rotors working on the decryption would be given the same set of rotors, as together they would emulate one Enigma. All the rotors were initially set to 'A', an arbitrary starting position, but the top row (the ones which rotate the most) would each be incremented by 1. So, the first set to A, the second to B, third to C and so on. This meant that the rotors on the Bombe were in the correct relative positions for each letter in a transmission on the Enigma (remember this would increment by 1 place with each letter), so simultaneously emulating the entire process of encrypting an entire message.

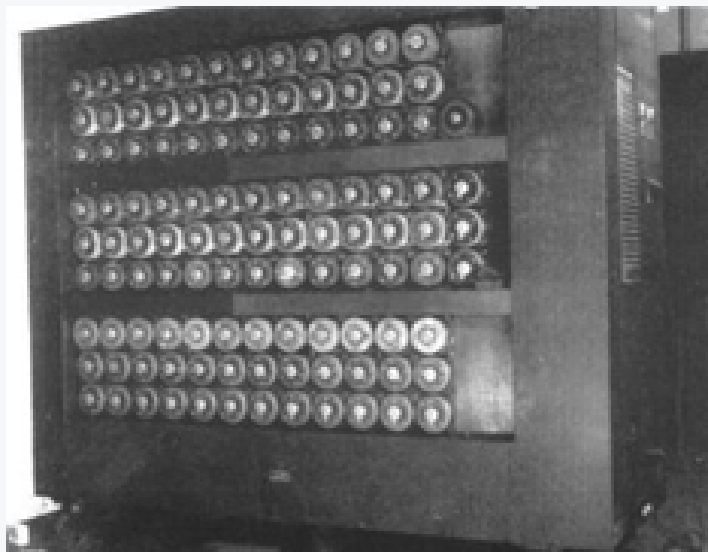


Figure 5: the original Bombe; all were destroyed and only replicas remain. See the rows of rotors, in vertical sets of 3.

Every letter had its own terminal on the Bombe, each terminal consisting of 26 wires which mapped onto every other letter, including itself. This acted as every possible Stecker setting at once, where the letter mapped onto itself is no switching.

This was then combined with our rotor settings and crib. If 'A' maps to 'B' at position 3 (after 2 rotations of the encrypting rotors), the terminal for 'A' is wired to the third set of rotors (emulating 3 rotations), which in turn is wired through to 'B'. This process is repeated for every letter in the crib, so the Bombe machine's circuits can be thought of as now corresponding to the previously described menu. All that must be found here is the correct rotor setting, and correct Stecker setting.

The machine simply cycles through the rotor settings, all 1,054,560 of them (including when the rotors are manually swapped around), but how can it know when to stop?

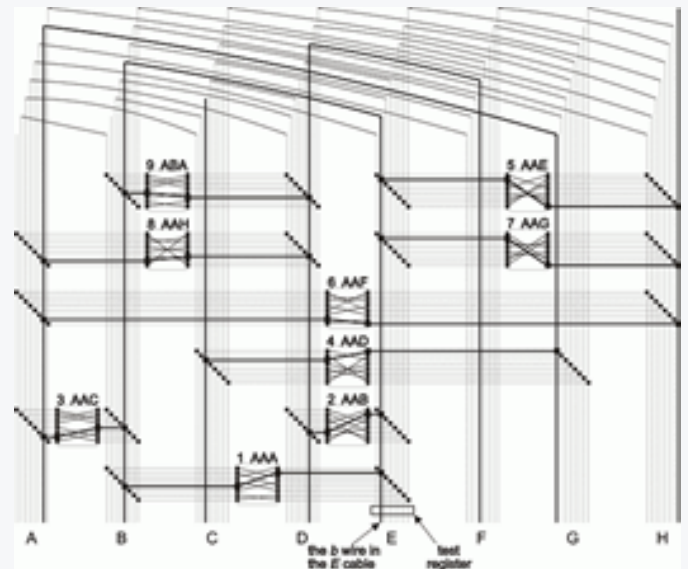


Figure 6: a diagram of a simplified 8-letter Bombe, showing a stopping condition (current in bold)

A current is applied to one letter's line within one terminal, usually chosen as the most common letters in the crib to aid the machine. As the current runs through the simulated Stecker connection, the hypothesis that these two letters are switched is tested. When the rotor order is correct, if the switch is correct every letter's terminal will receive one and only one output (as each letter can only match onto one other). If incorrect, the logical contradictions of letters mapping onto multiple others will flood the circuits of the Bombe. With a good enough crib, the current eventually enters all the circuits except those of the correct Stecker combination, which – as none of those combinations are mapped onto more than one letter – is completely isolated. And so, the correct combination is also revealed if 25 letters (all but 1) within every terminal have current. The Bombe can recognise when either of these situations occurs, and stops itself in that rotor position. [2]

Now the codebreakers have an initial rotor setting (by reading the position of the first set of rotors) and, by inspecting indicators within the machine to find the Stecker lines with current, the Stecker combinations which would work **for the crib**.

However, as mentioned earlier, the message could have actually been encrypted using different settings, and the found settings just coincidentally also worked for the given crib. To eliminate these false stops, codebreakers would use a mock Enigma machine to test the setting for the rest of the message (remember that without the initial setting an Enigma machine alone does not allow for decryption). If the decryption roughly resembled German, the correct setting had been found! The word 'roughly' is used here, as the crib may have left some letters out (and so the Stecker circuits did not link to them at all, meaning they would have to be manually found).

There is one final detail worth mentioning, and that is to do with how the first rotor incremented the second and the second the third within the Enigma. Each rotor had a 'Ringstellung', a metal ring on it which could be rotated, allowing the increment of the next (after 26 letters) to occur at any position of the rotor. This detail, which would normally make decryption even more tedious, was simply worked around in the design of the Bombe. By splitting up a crib shorter than 26 letters into two smaller cribs, each of which could be processed separately, it can be ensured that for at least one of them the rotation of the second wheel when setting the Bombe was not required (as in Enigma it would not rotate), and this detail could be brute forced out of the full message once the other settings had been discovered. With 676 combinations (26 for each of two rotors), this was still no easy task, and relied heavily on educated guessing.

Alan Turing calculated the statistical average number of stops before the correct one is found, for the number of letters in the menu which could be deduced. Another factor which affects the number of false stops is the number of 'loops' within the menu. In figure 4, AGK is an example of a loop of letters, meaning they each map onto each other at some point in the crib. The more loops contained within the menu, the rarer it is for the circuitry inside the Bombe to not produce any logic contradictions (and stop) for settings other than the correct one. The estimated number of stops ranges from 40,000 (for 8 letters and no loops) to less than 0.01 (for 16 letters and 3 loops). [3]

The ability of the Bombe is very clearly dependant on the ability of the codebreakers at Bletchley to produce a sufficient crib. The vast majority of these codebreakers were 'Wrens' – enlisted ranks or officers of the Women's Royal Naval Service – and it is as much thanks to their dedication and hard work as it is to that of Alan Turing that the work at Bletchley was successful.

The fundamental basics of computing were all present in the Bombe: sequence, iteration and selection. It procedurally ran through possible combinations (sequence), with mechanically rotors creating loops (iteration) and made us of electro-mechanical mechanisms acting as 'IF' statements to decide when to stop. By modern standards, it was incredible slow (making about **15 operations per second**, taking 20 minutes per set of rotors) – compared to apple's A14 chip capable of **11 trillion operations per second**. [4] But regardless of its speed, regardless of its original purpose, the Bombe was one of Turing's first computational machines to actually be built – and as such one of the first in the world.

It is often said that the Bombe helped shorten the war by many years, and by the end of the war roughly 155 had been constructed, with various improvements to increase efficiency or counter modifications to the German Enigma. However, its impact on computer science and technology is still evident today! A 'Turing machine' is a processing unit, first envisioned by Turing in the 1930s, designed to solve a logic problem; for every solvable logic problem there is, a Turing machine can solve it. Sound familiar? This is of course a description of the modern-day computer! The Bombe was a specialised machine, its operations were hard-coded in, and the 'programming' of codebreakers is more comparable to data entry for processing. Yet it was from this mechanical computer's implementation of selection and iteration a new machine named 'Colossus' was born, the world's first general purpose computer. And Colossus was made at Bletchley Park, by Alan Turing.

## References:

1) Carter, F., 2010. The Turing Bombe in Bletchley Park. [online] Rutherfordjournal.org. Available at: <<http://www.rutherfordjournal.org/article030108.html>> [Accessed 5 November 2020].

2) Ellsbury, G., 2003. The Enigma and The Bombe. [online] Ellsbury.com. Available at: <<http://www.ellsbury.com/enigmabombe.htm>> [Accessed 5 November 2020].

3) Carter, F., 2020. From Bombe 'Stops' to Enigma Keys. [ebook] p.3. Available at: <<https://web.archive.org/web/20100108030414/http://www.bletchleypark.org.uk/content/bombestops.pdf>> [Accessed 5 November 2020].

4) Apple Newsroom. 2020. Apple Unveils All-New iPad Air with A14 Bionic, Apple'S Most Advanced Chip. [online] Available at: <<https://www.apple.com/newsroom/2020/09/apple-unveils-all-new-ipad-air-with-a14-bionic-apples-most-advanced-chip/>> [Accessed 5 November 2020].

All images from  
<http://www.ellsbury.com/enigmabombe.htm>

# AN INTRODUCTION TO BLOCKCHAIN

## ARJUN JAVAGAL

A blockchain is a list of records, linked together for security using cryptography. It can be implemented in nearly any application, which requires the security and integrity of transactions, but is most notably used in cryptocurrencies. Since the 2008 Financial Crisis, cryptocurrencies have been on the rise, due to the assurance of security in capital they possess, as well as a decentralized system, where no central authority (like a bank) can hold your money. However, the promise made by cryptocurrencies can only be achieved by the technology behind blockchain.

There are two types of blockchain networks. A public network is a network where anyone can join, whereas a private network requires an invitation to be included. Generally private blockchain networks are used by companies or businesses, and public ones can be used between friends. Each member of a network has a list accompanying them, which is called a ledger. This is just a file which records, and stores transactions made on a blockchain network. All transactions on the network are stored on each independent ledger, to prevent the case of a member lying about a single transaction. After each transaction, the transaction is signed by the member sending money, using a digital signature. However, a digital signature can be easily forged, so to prevent overspending or counterfeiting, two algorithms, a signature and verification algorithm are required.

To do this, a DSA algorithm is used: this is a public-key encryption algorithm, which takes in the hashed message as well as the public key to create a signature. It is mainly used in the financial industry and in blockchain for transactions. Another algorithm (ElGamal) can also be used, but a DSA algorithm requires lower signature length (although having the same security level), which therefore means that DSA algorithms require less computation speed, and reduced storage space.

The algorithm works in these steps:

- The message hash is firstly computed (through SHA-256, for example)
- A random integer  $k$  is calculated, to make the algorithm non-deterministic (random), to increase security
- The signature is computed  $(r,s)$  where  $r$  is calculated from  $k$ , and  $s$  (signature) is computed from  $k + \text{message hash} + \text{private key}$

The verification algorithm needs to take in the hashed message, as well as the signature and public key, and it needs to output in a Boolean form whether the signature is verified properly. This can also be done by using a DSA signature verification algorithm. The same process can be followed with something called an RSA algorithm. This is a type of deterministic cryptosystem. The algorithm for signature production is as follows:

- The message hash is firstly computed
- The hash is encrypted by the private key
- The resulting integer is the signature algorithm

The RSA algorithm also contains a verification algorithm, which works like this:

- The message hash is firstly computed
- The message signature is decrypted with the public key
- The decrypted hash is compared with the hash of the signed message, to check whether it is correct

These 2 algorithms work differently (in producing signatures) but can be used to find and verify signatures. They are secure, as nobody can ever decrypt the message sent unless they know the private key, which should always be kept with the owner.

Furthermore, for verification, you can use something called Merkle Trees, which can be used to find if a certain transaction was in a block. To do so, you first find the hash of the transaction you want to find, and you then find the hash of the transaction before or after it. You combine the 2 hashes to form the first tree. Then, by knowing the next hash made in the next tree, you can combine those hashes to find the next hash and so on. The Merkle Root is the hash of all the hashes of the transactions: if the Merkle Root is equal to the block header, then this shows that the transaction was in that block.

After each transaction is completed, it is kept in a block. A block is just a list of transactions, with a proof of work. What is a proof of work? Say you have a block, and a special number: by applying SHA-256 to the block and special number, the resulting hash has to be compatible to the conditions set for the block - say for example, if block had to start with 60 zeroes, the resulting hash should have to have 60 zeroes in front (in reality, cryptocurrencies like Bitcoin change the conditions regularly, so that on average, it takes around 10 minutes to compute the proof of work, whereas other cryptocurrencies have different times). The advantage of having a proof of work system is that it is hard to compute the proof of work integer, but it is simple to verify it, as you can just compute SHA-256 with the proof of work in the block to check if it meets the conditions, to ensure the security of the block. However, what makes the fact that blockchain is so secure, is that the next block must contain the hash of the previous block. This creates a chain of blocks, hence blockchain. This means that if one of the blocks were somehow changed (the data inside had been changed somehow, or the blocks had swapped order), the hash of that block would completely change, which would also mean the hash of the next block would change - resulting in a chain reaction. To efficiently create blocks, Merkle Trees are again used. This can be done by lining up the transactions, to find the Merkle Root, or the hash of the hashes of the transactions, and then putting the resulting root hash into the block header. This is much more efficient as when mining, you only need to use the block header, instead of computing the Merkle root on the spot.

There are people all around the world willing to compute these proof of work integers for each block. They are known as Blockchain Miners, or just Miners. After transactions are broadcasted, these miners take in the transactions to create a block. Then with that, they try to find the appropriate proof of work, so that the hash of the entire block satisfies the certain conditions. Out of all the miners, whoever computes the proof of work the fastest, receives a reward. For example, in Bitcoin, this reward would be 6.25 B. After the proof of work is computed, it is then broadcasted back to the participants in the network that the transaction was sent out of to be stored in the blockchain.

However, there are instances in which 2 chains with differing transaction histories are being broadcasted to a member in the network. If this is the case, the longer chain is always chosen. This is called the Longest Chain Rule. The reason for this can be shown practically: say person one sends 100 bitcoins to person two but does not broadcast that transaction to others in the network. This means that everyone else thinks that person one has not sent 100 bitcoins. After the transaction has been made, the only way person one can fool everyone in the network is by finding a valid proof of work faster than other miners. This can occur - but person two can still hear blocks made by different miners. This means that person one has to continually produce blocks faster than the miners, which over time becomes harder and harder, so person one outputs less blocks than the miners, meaning with this new protocol, person two should trust the longest block, which comes from the miners. This shows how by using this system, the member can efficiently authenticate the blocks, and which blockchain to trust.

However, there are flaws in the blockchain system. One of the biggest problems is the 51% Problem, where a group of miners take control of over 50% of the network's hashing power. This is a big problem as that group have the ability to interrupt the recording of new blocks by stopping other miners, meaning they can choose which blocks they can record. As well as that, this attack allows miners to double spend their coins.

Double spending is when members can spend the same currency twice by duplicating the transactions. In a practical example, say an attacker sends coins to person one, whilst sending the same coins to person two. The attacker sends the person one transaction, to the rest of the members in the network, however he/she does not do that for the person two transaction. Since the attacker has so much computational power, he/she can compute all of person two's blocks, to make the transaction seem valid, by creating a longer blockchain than the other miners for validation, but leave out person one's blocks. This means the transaction sent to person one will be invalid so they receive no money, and the attacker would be paid for his service externally. An example of this occurred on Krypton and Shift, 2 blockchains based on Ethereum, in August 2016. The best way to prevent 51% attacks are to decentralize miners, so they cannot collectively benefit from each other's computational power.

In summary, a blockchain is just a list of records containing blocks. These blocks contain transactions, which are verified by the recipient/sender of the transaction. The block is then hashed to find the proof of work by miners and is then broadcasted back to members of that specific network. Like said beforehand, blockchain is not just used in cryptocurrencies. In fact, it can be applied in anything requiring data to be secure.

# WHY YOU SHOULD LEARN TO CODE

## NAVANEETH MADHAVAN

Coding is the process of writing a specific set of instructions that a computer or device can understand, in order for it to perform a certain activity. The programmer essentially teaches the computer step by step, how to carry out a task. Computers can only detect the flow or absence of electricity in the billions of transistors that make up their processor (see figure 1). This is represented by 1s and 0s in binary – one being the flow of electricity and zero the absence. Code in binary can be written in binary machine code which is a 'low-level language'. However, writing code in a low-level language would be too cumbersome and make it difficult for programmers to understand each other's work and collaborate. Due to this, most work is done in 'high-level' languages (i.e. Java, Python and Ruby) that are much more similar to human languages (i.e. English). High-level code can then be translated by a compiler or interpreter into the binary machine code that a computer can understand. By learning to code, we are able to utilise billions of transistor combinations, consequently harnessing the ability to make computers perform very powerful tasks. Familiarising yourself with this relatively modern discipline will most certainly lead to a sea of possibilities.

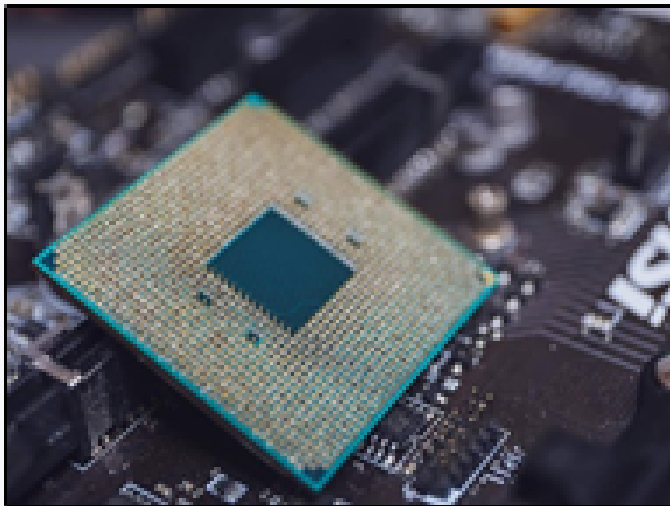


Figure 1: transistors in a processor

### The age of technology and automation

We are now in the midst of the age of technology. Coding is becoming more and more relevant. It is all around us. From the phone sitting in your pocket to the aeroplane flying above your head – pretty much all machines are integrated with code. Think about the countless times you have surfed the internet, mindlessly scrolled through social media, but have you ever stopped to think about how it all works? Underneath the hardware, oblivious to the user, there are millions of lines of binary intricately weaved into machines. By gaining an understanding of code, you will be more aware of how technology really influences our lives.

In the last 50 years there has been a rapid increase in automation. Machines are slowly taking over the service sector. For instance, McDonald's now have electronic kiosks that allow you to order without the need of cashier (see figure 2). No human contact is necessary. Furthermore, with the rise of digital companies such as Amazon, having to actually leave your house to go shopping is becoming unnecessary, rendering many small, niche stores obsolete.

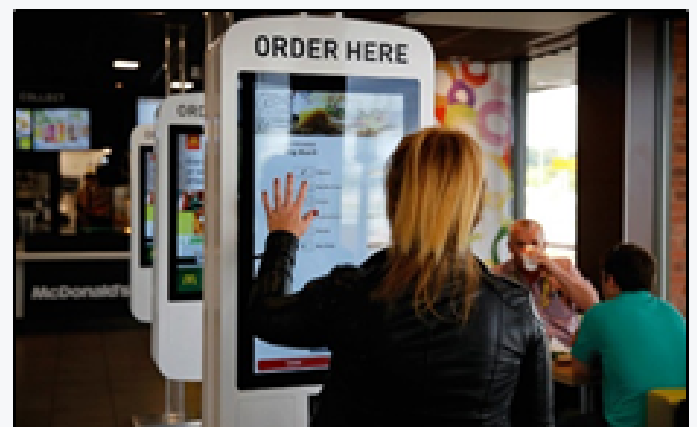


Figure 2: A McDonald's kiosk

Due to the emergence of machine learning (a form of artificial intelligence), it is not only basic service jobs that are being taken over by machines. Middle management jobs such as admin can be replaced by computers. For instance, a company in San Francisco offers a project management software (dubbed 'iCEO') for big corporations that eliminates the need for these middle management positions. The software first reviews the company to see which specific jobs can be automated. It then studies how the workers complete those jobs, learning to do them through machine learning. In short, the employees teach machines to replace them.

The rather frightening example above highlights the reality of how most jobs done by humans will soon be replaced by technology. By 2030, a shocking 73 million US jobs are expected to be automated. The problem is getting very real. However, there is one job that is exempt from this – computer programming. In the digital age, there will always be a need for people who can tame and control the machines that society relies on. Specialists who can write and update the software that computer systems function on are priceless.

## **Jobs and work**

Currently there is a lack of people to fill up software engineering jobs. A proficient coder will therefore be high in demand. They will never be short of a job. So, what tasks does this role consist of? As a software engineer, you will have to write code for a company such as Google or Facebook. You may also have to update their software and fix bugs to make sure it is all working well. There are many benefits of having a job in the field of software engineering. The first is that the pay is very good (the average software engineer's annual salary in London is £51,000, compared to national general average salary of £31,000). Another benefit is that you have the choice to work at home (since most tasks can be done on a laptop). This is great since you can work with privacy in your own comfortable environment and not have to worry about rushing to work every morning. Hours are also very flexible meaning part-time jobs are possible (these are ideal for students looking to make some extra money on the side).

Furthermore, most software engineers enjoy their jobs greatly and find it extremely stimulating – unearthing the answer to a problem you first thought impossible yields a sense of satisfaction unmatched by any!

It might surprise you that software engineering is not the only job that involves coding. There are many other jobs in various sectors that will require an understanding of code. For example, software is a large component of many machines in the engineering sector. As well as this, many scientists use specialist software in order to conduct experiments with precision and accuracy. Although coding may not be the main focus of these jobs, it is still a key aspect. Consequently, a strong background in coding will definitely give you an edge in the field you choose to pursue a career in.

With the skills you obtain from coding, you do not necessarily have to work for someone but rather can work for yourself. You could code and sell software, design a website or create and sell an app on the app store. Your humble start-up may have the potential of turning into a billion-dollar company – joining the likes of Facebook and Google. There are endless business ventures to choose from. An advantage of selling a digital product is that you will not need a large capital to get the business running since you are not physically manufacturing an item. Producing the software product will take less time and resources than if it was physical/hardware product and most of the logistics by can be handled by yourself. These benefits make it much easier to make your great ideas go a long way.

## **Other benefits of learning to code**

Amongst the financial benefits of learning to code, there are benefits of coding as a hobby. As mentioned earlier, the thrill of problem solving is exhilarating meaning coding can be an extremely rewarding task. Furthermore, you are constantly using your brain and thinking abstractly – due to this, studies show that people who code are less likely to suffer mental illnesses such as dementia. Coding many projects will teach you how to think. In many cases you are modelling the real world, learning various algorithms in detail.

This forces you to look at problems from various angles, thus promoting rational and critical thinking.

In addition to this, you can implement coding into your life for more practical applications. For example, you can program a raspberry pi – a pocket sized computer – to make it serve a multitude of purposes from a home security system to an electronic doorbell (see figure 3). As well as this, you can design apps that may make your life more productive – i.e. an app that reminds you what homework you have and tells you when to do it. Furthermore, you can code games for the entertainment of you and your friends. This will be a great way to pass the time and share your skills to others.



*Figure 3: Raspberry Pi security camera*

## **How to begin your coding journey**

Now I know what you are all thinking – how can I begin my coding journey? The first thing to remember is that learning to code is not something that will happen overnight – it is a steady process that takes hours of work and dedication. However, that does not mean that it is not worth your while – anything fruitful requires hard work. Although the journey may be tough at times make sure to persevere and overcome any challenges with determination.

When starting off, the sheer number of programming languages to choose from can be overwhelming. Before choosing what language you want to learn, make sure you have a clear idea of what your goal is.

Every language has different uses and will be appropriate for different tasks. Here is a helpful list: for iOS apps choose Swift; for Android apps choose Java or Kotlin; for websites choose JavaScript, HTML or CSS; for data, engineering, or science choose Python, R, MATLAB; for game development choose C++ or C#. Right now, do not worry too much about what language you are going to learn first since it is most likely you will learn many different languages, for many different use cases, throughout your career. After you have studied your first language, it is not difficult to learn another, since all programming languages have similar fundamentals. If you are still unsure what language to begin with, I would recommend python – it is simple, straightforward and gives you a solid grasp of the basics of coding.

After you have chosen a language and downloaded it, there are many resources and techniques available to help enhance your understanding. It is always a good idea to purchase a book that teaches you the key ideas. You could also watch tutorials on YouTube or find a course to complete online. If you are stuck at any point, you can share queries on sites such as stack-overflow that are used by a large community of coders. A great way to accelerate your learning is by doing projects that are can be engaging and interesting. For instance, you could start off with something simple like a tic-tac-toe game. Collaboration sites such as GitHub are a great platform to share your digital masterpieces. You can receive useful feedback as well as helping a community of programmers. Once you have become proficient in the language, you may want to test your skills on sites such as HackerRank that promote problem solving.

A Computer Science degree can be really useful if you wish to pursue a career in code. Although this is not necessary for all software related jobs, it will be beneficial since you will be taught the maths and logic beneath the code itself. This will give you a deeper understanding of coding priming you well to tackle real world tasks. Learning various programming languages will definitely become less of a challenge.

## Possible negatives of coding and misconceptions

Throughout this article, I have shed light up the many positives of coding, however, there are some negatives that are important to discuss.

Programming is a sedentary job. Sitting down for too many hours at a time will have a significant effect on your posture. To avoid this, make sure that you have your desk ergonomically set up with your back straight, elbows level with the table and screen positioned at eye level. This will ensure that your body is not positioned awkwardly.

Furthermore, make sure to take breaks to stand up and stretch feet once every hour or so that you are not fixed to your chair for too long. Another issue is that many programmers get too fixated on solving certain code – sacrificing their physical, mental and social well-being for work. They may even give up sleep being too engrossed in a problem. Although there is nothing wrong with working hard, it is vital to have a balanced life.

There are also many misconceptions society has on coders. Many people consider programmers to be socially awkward. This is however not always the case. Even though software engineering involves lots of solo work, collaboration with others is a key aspect. Programmers need to work together on large projects in order to come up with a high-quality code. Teamwork is vital since others may be able to think of better solutions in various situations and see problems from different angles. Another misconception is that coding is considered to be too difficult and not suitable for all people. This is most certainly not true. As said before, although it can be challenging at times, a mixture of hard work and determination will ensure success.

## Conclusion

As discussed, there are numerous benefits of coding. Whether you are looking for a serious high paying job or a fun hobby on the side – coding is the one. Join the ranks of legends such as Bill Gates and Mark Zuckerberg who started their journey into programming as young teenagers.

Will you too be the owner of a billion-dollar tech-empire? What will you create? Where will coding take you?

## Bibliography:

Project management software:

<https://www.linkedin.com/pulse/machine-replacing-project-managers-rajesh-bhattad->

Jobs to be automated by 2030:

<https://www.ttnews.com/articles/automation-could-kill-73-million-us-jobs-2030>

Average software engineer's annual salary in London:

[https://www.glassdoor.co.uk/Salaries/london-software-engineer-salary-SRCH\\_IL.0,6\\_IM1035\\_KO7,24.htm](https://www.glassdoor.co.uk/Salaries/london-software-engineer-salary-SRCH_IL.0,6_IM1035_KO7,24.htm)

National general average salary:

<https://www.gq-magazine.co.uk/article/average-uk-salary>



# COMPUTER SCIENCE SOCIETY

COMPUTER SCIENCE SOCIETY MEETS EVERY MONDAY IN IT2 AND IS AIMED AT GOING BEYOND THE GCSE AND A-LEVEL CURRICULUM TO ENCOURAGE INTEREST AND BROADEN KNOWLEDGE IN ALL ASPECTS OF COMPUTER SCIENCE. WE HOST TALKS AND EVENTS ON A WIDE VARIETY OF TOPICS OPEN TO ALL YEARS TO FOSTER A WIDE RANGE OF OPINIONS. COMPUTER SCIENCE IS A DYNAMIC, RAPIDLY DEVELOPING PART TO MODERN LIFE AND HAS MANY TRANSFERABLE PROBLEM-SOLVING SKILLS THAT ENABLE YOU TO SOLVE COMPLEX CHALLENGES AND MAKE A POSITIVE DIFFERENCE IN THE WORLD. OUR AIM FOR THE SOCIETY IN THE UPCOMING YEAR IS HIGHER INCLUSIVITY - WE WANT TO MAKE SURE EVERYONE IS INVOLVED, REGARDLESS OF PRIOR KNOWLEDGE OR WHAT YEAR GROUP YOU ARE IN - THERE WILL BE SOMETHING FOR EVERYONE!

---



# **HYPertext**

ISSUE TWO • DECEMBER • 2020



---

**THANK YOU FOR READING!**